Week10-Downsampling and concatenation

Macarena López-Mayorga

2026-07-28

Why Downsample?

There are various reasons why we might want to downsample (subset our .fcs files to a certain number or percentage of cells), especially in context of unsupervised analysis.

Traditionally, one of the main ones is limited computational resources. Rapid Access Memory (RAM) was often in limited quantity, especially compared to the size of .fcs files. When working with a large dataset, downsampling allowed for more equal representation across all acquired files to be accounted for in the subsequent analysis phase, without maxing out the available RAM and triggering the software to crash out due to lack of memory. This is particularly the case for some unsupervised clustering and dimensionality reduction algorithms, that are trying to differentiate how similar or different all the cells within the analysis are from each other.

Separately, some statistical analysis methods primarily rely on counts. Unlike frequency, which partially standardizes the comparison by leveraging against the parent gate, methods that rely on counts for their statistic may be similarly assisted when a defined number of cells at a designated gate are utilized.

Regardless of reason, we will need to figure out a few logistics when implementing a downsampling strategy in R. We will first figure out the process using a single specimen, leveraging what we learned within the GatingSet lesson to be able to specify our gate of interest, and then leverage the resulting code to implement a function that can be used to iterate through all the files within the gating set.

Setup

Load fcs files

Before we can downsample, we will need to have our .fcs files brought into R. We consequently repeat the loading in process that we have been seeing fairly regularly throughout the first section. This week, we will be working with some "larger" spectral .fcs files (since we will need to downsample). We are still limited by GitHub's cap on max file size (5 MB), so if you want to use your own data, please feel free to substitute in the file path to your own .fcs files storage location.

```
StorageLocation <- file.path("C:/Users/Macarena/MacarenaLM/notes/
week10-Downsampling and concatenation/data")
fcs_files <- list.files(StorageLocation, pattern=".fcs", full.names = TRUE)
fcs_files
```

[1] "C:/Users/Macarena/MacarenaLM/notes/week10-Downsampling and concatenation/data/2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs" [2] "C:/Users/Macarena/MacarenaLM/notes/week10-Downsampling and concatenation/data/2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs" [3] "C:/Users/Macarena/MacarenaLM/notes/week10-Downsampling and concatenation/data/2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs"

```
library(flowWorkspace)
library(flowGate)
library(dplyr)
library(ggplot2)
library(purrr)
```

```
SFC_cytoset <- load_cytoset_from_fcs(fcs_files, truncate_max_range = FALSE,
transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoset)
SFC_Parameters <- colnames(SFC_GatingSet)
FluorophoresOnly <- SFC_Parameters[!stringr::str_detect(SFC_Parameters,
"FSC|SSC|Time")]

Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
     pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 3 samples

Gate

Once our data is in a GatingSet, we can add some general gates for the subsets using the flowGate package

```
GatingTable <- tibble::tribble(
  ~filterId,    ~dims,                          ~subset,
  "singlets",   list("FSC-A", "FSC-H"),          "root",
  "live",       list("FSC-A", "Zombie NIR-A"),   "singlets",
  "Tcells",     list("CD3", "CD45"),             "live",
  "CD4+",       list("CD8", "CD4"),              "Tcells",
  "CD8+",       list("CD8", "CD4"),              "Tcells",
  "DN",         list("CD8", "CD4"),              "Tcells",
)
```

```
flowGate::gs_apply_gating_strategy(SFC_GatingSet, gating_strategy = GatingTable)
```

Listening on http://127.0.0.1:5082 done!

Listening on http://127.0.0.1:5082 done!

Listening on http://127.0.0.1:5082 done!

Listening on http://127.0.0.1:5082 Warning: Error in if: argument is of length zero 151: getDims 150: catch 149: value[[3L]] 148: tryCatchOne 147: tryCatchList 146: tryCatch 145: do 144: hybrid_chain 143: 121: drawReactive 108: renderFunc 106: output$plot$18 : $shiny$ :: $runApp7$: $.f3$: $pmap_2$: $purrr$ :: $pmap1$: $flowGate$ :: $gs_apply_gating_strategy Warning$: $Error in if$: $argument is of length zero 223$: $prepare Plot 222$:< $reactive$ > 200 : $FPlot 199$: $renderPlot 106$: $output$plot1 8: shiny::runApp 7: .f 3: pmap_ 2: purrr::pmap 1: flowGate::gs_apply_gating_strategy done!

Listening on http://127.0.0.1:5082 done!

Listening on http://127.0.0.1:5082 done! [[1]] [[1]]$Gate Polygonal gate 'singlets' with 25 vertices in dimensions FSC-A and FSC-H

[[1]]$Bins [1] 256

[[1]]$Scaling [1] "unused"

[[2]] [[2]]$Gate Rectangular gate 'live' with dimensions: FSC-A: (956353.75462107,3862903.5) Zombie NIR-A: (1777.4200266594,2870.4400131676)

[[2]]$Bins [1] 256

[[2]]$Scaling [1] "unused"

[[3]] [[3]]$Gate Polygonal gate 'Tcells' with 20 vertices in dimensions Spark Blue 550-A and Spark Blue 574-A

[[3]]$Bins [1] 256

[[3]]$Scaling [1] "unused"

[[4]] [[4]]$Gate Polygonal gate 'CD4+' with 14 vertices in dimensions BV650-A and BUV805-A

[[4]]$Bins [1] 256

[[4]]$Scaling [1] "unused"

[[5]] [[5]]$Gate Polygonal gate 'CD8+' with 17 vertices in dimensions BV650-A and BUV805-A

[[5]]$Bins [1] 256

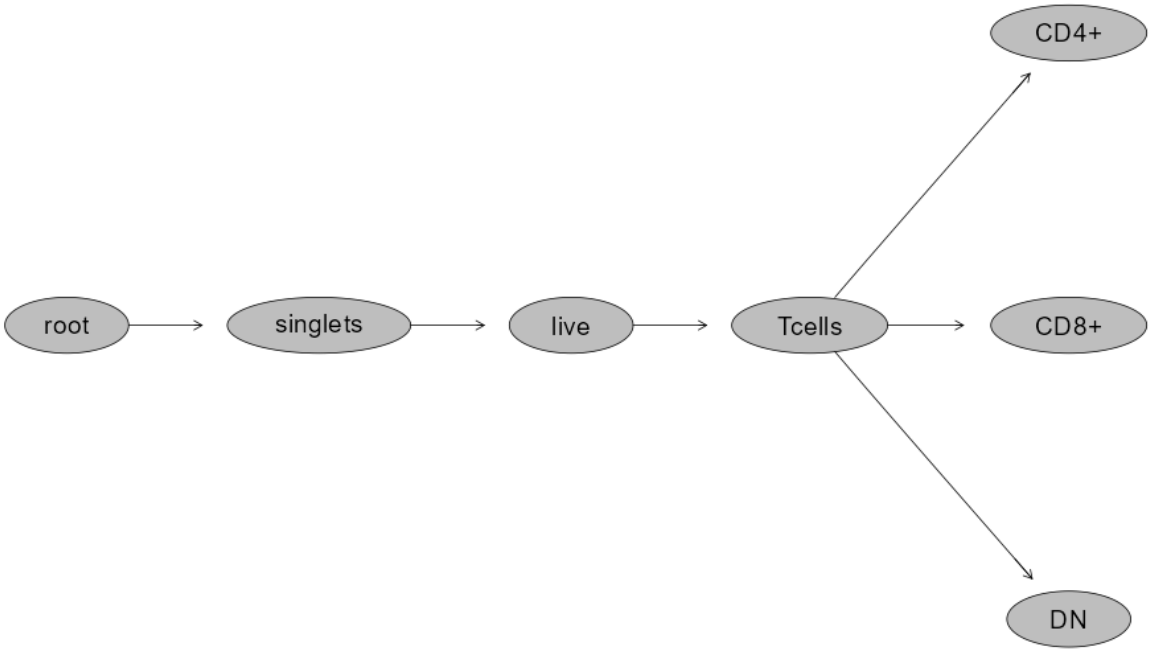
[[5]]$Scaling [1] "unused"

[[6]] [[6]]$Gate Polygonal gate 'DN' with 11 vertices in dimensions BV650-A and BUV805-A

[[6]]$Bins [1] 256

[[6]]$Scaling [1] "unused"

```
plot(SFC_GatingSet)
```




```
StorageLocation <- file.path("C:/Users/Macarena/MacarenaLM/notes/
week10-Downsampling and concatenation/data/saving GatingSet")
save_gs(SFC_GatingSet, StorageLocation)
```

Done To reload it, use 'load_gs' function

```
StorageLocation <- file.path("C:/Users/Macarena/MacarenaLM/notes/
week10-Downsampling and concatenation/data/saving GatingSet")
SFC_GatingSet <- load_gs(StorageLocation)
```

Retrieving counts

Let's quickly check to see what specimens we will be working with for this dataset.

```
pData(SFC_GatingSet)
```

2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs

We have so far gated for the three main T cell populations in cord blood (CD4+, CD8+ and Double-Negative (CD4-CD8-)). Considering that in cord blood mononuclear cells, the abundance of these subsets may vary a bit by donor, we want to make sure we downsample a number of cells that will result in each individual specimen providing a relatively similar contribution of cells to the final .fcs file we end up creating.

To get better context before diving in, lets go ahead and retrieve, then plot the statistics data for our gates so that we have an idea of what we are working with (and get some additional dplyr and ggplot2 practice in, as it has been a while since Week 4 and Week 06).

```
Data <- gs_pop_get_count_fast(SFC_GatingSet)
```

Looking at our retrieved data, the gate names are showing up as full file.paths.

```
Data$Population
```

[1] “/singlets” “/singlets/live” “/singlets/live/Tcells” “/singlets/live/Tcells/CD4+” [5] “/singlets/live/Tcells/CD8+” “/singlets/live/Tcells/DN” “/singlets” “/singlets/live”
[9] “/singlets/live/Tcells” “/singlets/live/Tcells/CD4+” “/singlets/live/Tcells/CD8+” “/singlets/live/Tcells/DN”
[13] “/singlets” “/singlets/live” “/singlets/live/Tcells” “/singlets/live/Tcells/CD4+” [17] “/singlets/live/Tcells/CD8+” “/singlets/live/Tcells/DN”

Let’s abbreviate them for simplicity using the basename() function.

```
Data$Population <- basename(Data$Population)
Data
```

	name	Population	Parent	Count	Pa
	<char>	<char>	<char>	<int>	

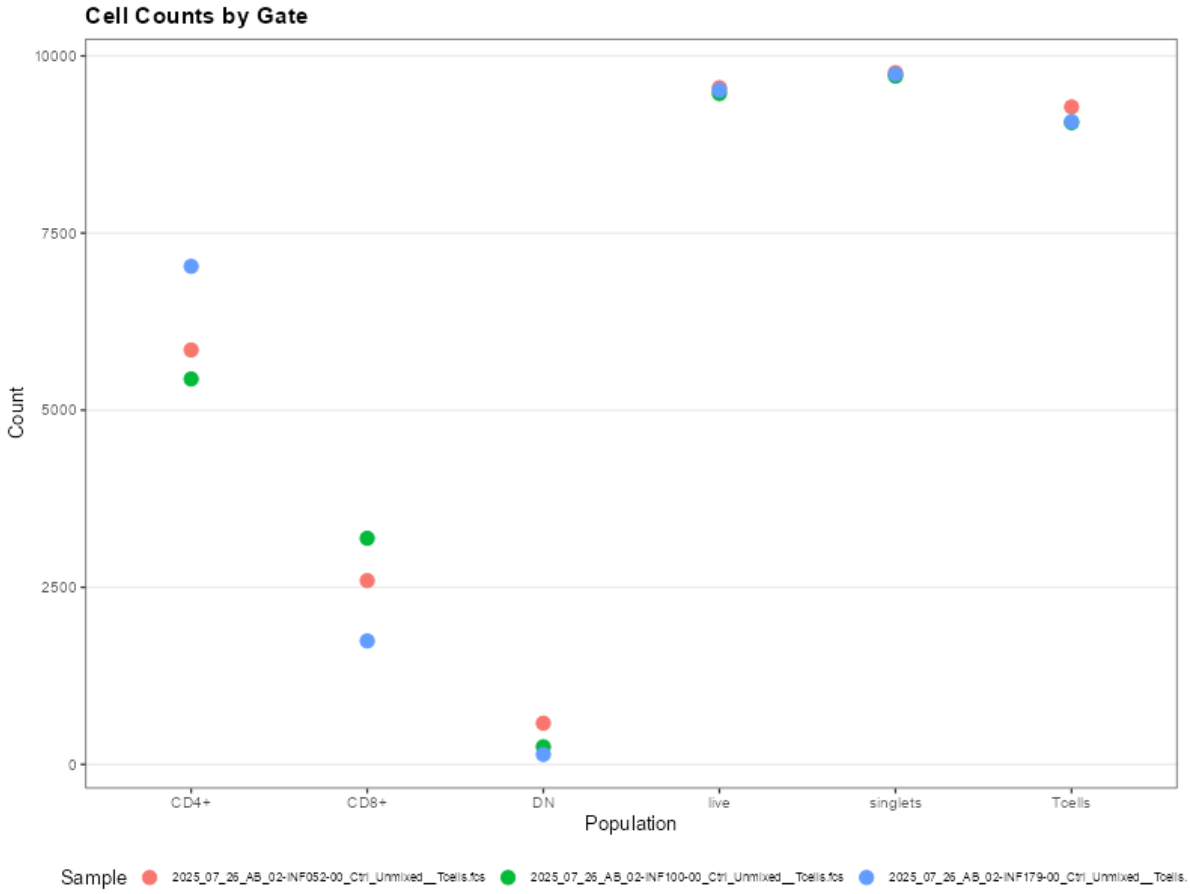
1: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs singlets root 9764 10000 2: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs live /singlets 9552 9764 3: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs Tcells /singlets/live 9280 9552 4: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs CD4+ /singlets/live/Tcells 5850 9280 5: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs CD8+ /singlets/live/Tcells 2594 9280 6: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs

DN /singlets/live/Tcells 582 9280 7: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs
singlets root 9715 10000 8: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs live
/singlets 9469 9715 9: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs Tcells
/singlets/live 9059 9469 10: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs
CD4+ /singlets/live/Tcells 5440 9059 11: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs
CD8+ /singlets/live/Tcells 3191 9059 12: 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs
DN /singlets/live/Tcells 248 9059 13: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
singlets root 9744 10000 14: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
live /singlets 9516 9744 15: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
Tcells /singlets/live 9072 9516 16: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
CD4+ /singlets/live/Tcells 7030 9072 17: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
CD8+ /singlets/live/Tcells 1742 9072 18: 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs
DN /singlets/live/Tcells 141 9072

With this bit of cleanup done, lets plot them with ggplot2

```
Plot <- ggplot(Data, aes(x = Population, y = Count, color = name)) +
  geom_point(size = 4) +
  labs(
    title = "Cell Counts by Gate",
    x = "Population",
    y = "Count",
    color = "Sample"
  ) +
  theme_bw(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold"),
    legend.position = "bottom",
    legend.text = element_text(size = 8),
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank()
  )

Plot
```



As we have encountered on a couple of the prior sessions, we can use the plotly package ggplotly() function to convert our static ggplot2 plots into interactive ones, which are useful in this context.

```
plotly::ggplotly(Plot)
```

Looking at our gated T cell populations across specimens, we can see that in general, while each specimen has similar number of live T cells, there is a little more variability when it comes to the individual T cell subsets. We will revisit this later on as we build out the function logic.

Building Downsample

Broad Idea Sketch

Now that we have our dataset pre-requisites assembled within our local environment, lets start by planning out what we will need in order to assemble a downsampling function, at least in terms of inputs and what it will ideally return as outputs.

We are going to be starting off with our gated GatingSet, and similar to what we did during Week 09 with the CellConcentration() function, iterate through the individual GatingHierarchies using the purrr packages map() function.

Once an individual .fcs file ends up within our new function, we will need to extract out the exprs data (where measurements for individual cells are stored). From this original data, we will need to downsample (i.e. subset) a designated number of cells which correspond to individual rows (while also accounting for several possible exceptions we might encounter).

This modified exprs data then needs to be returned to the .fcs file, maintaining the rest of the parameter and description metadata intact so that it remains recognizable as a standard .fcs file. We would also want to be able to export out the .fcs file with modified name parameters so that we can distinguish the downsampled version from the original .fcs file, to avoid accidentally overwriting our original.

So, visualizing ahead, at the end of the iteration, we would end up with three new .fcs files, containing our target number of downsampled cells originating for our respective gate of interest.

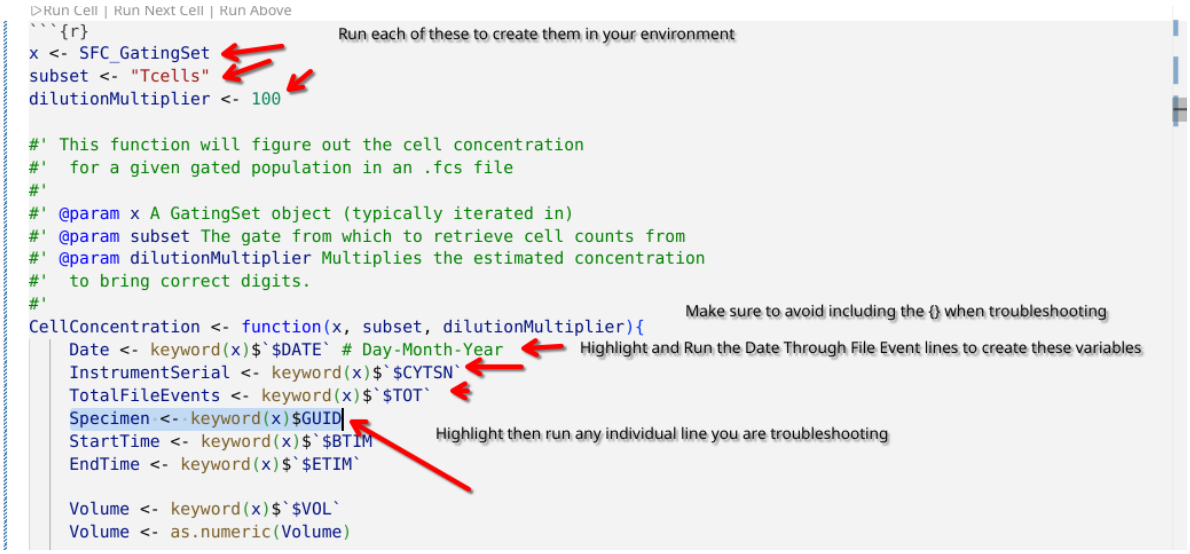
With this rough sketch worked out, lets dive in.

Initial Skeleton

Getting started, lets go ahead and establish our initial function, as well as add elements of the roxygen2 skeleton for documentation. We will provide our first argument as "x", which will serve as our standin for the individual .fcs file being iterated in via purrr.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#'
Downsampling <- function(x){
  # Our code will go here
}
```

When function building, highlighting and running (via Ctrl/Command + Enter) individual arguments being provided to the function can be helpful as you are writing it. These variables end up being created as objects in your environment (appearing under the variables tab in the right-secondary side-bar), and are available for use in troubleshooting and debugging. Here is an example of how highlight lines within the function that you want to run/troubleshoot would appear as.



Remembering back to last week, we remember that when we iterate a GatingSet object, we end up with a GatingHierarchy containing a single .fcs file, similar to if we had used [[]] on the GatingSet.

```
x <- SFC_GatingSet[[1]]
x
```

Sample: 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs GatingHierarchy with 7 gates

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#'
Downsampling <- function(x){class(x)}
```

```
class(x)
```

[1] “GatingHierarchy” attr(,“package”) [1] “flowWorkspace”

In this case, both are returning the same class of object, so we have correctly set up our function and outside argument standins correctly. Lets proceed to modify our the internals.

From the entire .fcs file, we will need to subset out the underlying data corresponding to our gated population of interest. This is similar to the code we used last time for CellConcentration, so we can quickly relocate that code from the respective lesson, then copy-and-paste it into our new function within the {}.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#'
Downsampling <- function(x){
  EventsInTheGate <- gs_pop_get_data(x, subset)
}
```

One thing to remember, the code within the function is only able to see variables that we pass in to it, which is done via arguments (that are present within the "()"). So to get gs_pop_get_data() to run successfully, we will need to add "subset" as Downsampling's second argument, or we will not be able to isolate the data associated for our respective gate.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#'
Downsampling <- function(x, subset){
  EventsInTheGate <- gs_pop_get_data(x, subset)
}
```

Having made a change to the function, we need to re-run the code-block above, so that the changes we have made to our function are reflected within our environment. Once this code-block has been rerun, if we check under the variables tab we can see the information detailed for our function has changed. Likewise, if we run our function using our actual arguments, we can see that the returned object has now changed from returning the class() output we had as a placeholder to the returned 'cytoset' object from gs_pop_get_data()

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

[[1]] A cytoset with 1 samples.

column names: Time, SSC-W, SSC-H, SSC-A, FSC-W, FSC-H, FSC-A, SSC-B-W, SSC-B-H, SSC-B-A, BUV395-A, BUV563-A, BUV615-A, BUV661-A, BUV737-A, BUV805-A, Pacific Blue-A, BV480-A, BV570-A, BV605-A, BV650-A, BV711-A, BV750-A, BV786-A, Alexa Fluor 488-A, Spark Blue 550-A, Spark Blue 574-A, RB613-A, RB705-A, RB780-A, PE-A, PE-Dazzle594-A, PE-Cy5-A, PE-Fire 700-A, PE-Fire 744-A, PE-Vio770-A, APC-A, Alexa Fluor 647-A, APC-R700-A, Zombie NIR-A, APC-Fire 750-A, APC-Fire 810-A, AF-A

cytoset has been subsetted and can be realized through 'realize_view()'.

Accessing exprs

With our function building underway, gs_pop_get_data() returns to us a "cytoset object" of length 1. Remembering back during Week 03, we were able to use exprs() on a flowFrame object to retrieve the underlying MFI measurement data that we are interested in. Let's try running it in this context and see if this will similarly work.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#'
Downsampling <- function(x, subset){
  EventsInTheGate <- gs_pop_get_data(x, subset)
  MeasurementData <- exprs(EventsInTheGate)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

Error in `map()`: In index: 1. Caused by error: ! unable to find an inherited method for function 'exprs' for signature 'object = "cytoset"'

Show Traceback

Judging by the error message, the exprs() function has no idea what do with a 'cytoset' object. With a little (or quite a lot) of investigation within the exprs help file and the flowWorkspace vignette, we see that the expected object being passed to the function is a mismatch for class, i.e. we are a level too high up in the hierarchy. Rather than passing a cytoset, we need to be at cytoframe level (individual object rather than a set) to successfully retrieve the exprs-associated data. Fortunately, dropping down to an individual unit is similar to other list style objects, requiring us to modify the code by placing [[1]] next to our cytoset variable inside the

function (EventsInTheGate). After updating the function (and re-running it), we can pass our data and check the output.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#'
Downsampling <- function(x, subset){
  EventsInTheGate <- gs_pop_get_data(x, subset)
  MeasurementData <- exprs(EventsInTheGate[[1]])
}
```

```
Data <- map(.x=SFC_GatingSet[1], subset="Tcells",
 .f=Downsampling)
head(Data[[1]], 3)
```

```
   Time    SSC-W  SSC-H    SSC-A    FSC-W   FSC-H   FSC-A  SSC-B-W SSC-B-
H  SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A BUV737-A BUV805-A Pacific Blue-
A

[1,] 88303 800625.8 412920 550990.7 745687.8 1207102 1500202 767241.9 336298 430036.5
2258.554 2967.613 1925.955 2098.853 3232.693 1879.053 2059.758 [2,] 151951 721600.7 522319
628176.2 713266.4 1364125 1621641 709783.5 365916 432868.6 2937.244 2664.265 2098.485
1966.986 3253.292 3515.342 2120.267 [3,] 225780 747106.5 305002 379781.6 678442.0 899363
1016943 694236.2 317438 367294.9 3243.016 2398.396 2251.842 2118.144 3098.917 3549.486
2179.407 BV480-A BV570-A BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor
488-A Spark Blue 550-A Spark Blue 574-A RB613-A RB705-A RB780-A PE-A [1,] 2112.291
1701.588 3548.215 3713.888 2058.872 2508.425 1867.927 2302.473 3419.844 3225.375 2187.869
3421.698 2137.918 2676.039 [2,] 2107.440 2218.929 3529.080 2071.707 2316.910 1962.467
2345.562 2223.587 3478.904 3267.592 2071.955 3753.434 2118.650 2939.783 [3,] 2134.529
1849.710 3272.614 1848.897 1957.577 2104.743 1955.958 2390.761 3435.635 3201.560 2368.059
3299.392 2021.984 2868.770 PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A PE-Fire 744-A
PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A Zombie NIR-A APC-Fire 750-A
APC-Fire 810-A AF-A [1,] 2167.108 2276.323 3108.547 2156.676 1954.386 2203.288 2097.104
2124.087 2319.684 3290.837 3289.486 2878.465 [2,] 2391.270 2032.860 2862.195 2188.545
2107.110 2056.971 2252.323 2084.971 2145.456 3337.221 3100.115 2868.599 [3,] 2305.990
2083.745 2462.668 2227.207 1976.180 2335.905 2157.208 2105.890 2191.646 3302.940 2974.669
2720.577
```

Success, we have successfully retrieved the underlying data for T cells! (finally!) But let's quickly do a sanity check, and make sure that the numbers we are retrieving make sense.

We can do this by adding summary() function to summarize the distribution for each of our MeasurementData columns.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#'
Downsampling <- function(x, subset){
  EventsInTheGate <- gs_pop_get_data(x, subset)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  summary(MeasurementData)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

[[1]] Time SSC-W SSC-H SSC-A FSC-W FSC-H FSC-A SSC-B-W SSC-B-H
Min. : 19 Min. : 650411 Min. : 99153 Min. : 149656 Min. :617511 Min. : 775895 Min. : 940884 Min. : 626911 Min. : 104214
1st Qu.:223170 1st Qu.: 746861 1st Qu.: 376063 1st Qu.: 484396 1st Qu.:712088 1st Qu.:1161711 1st Qu.:1400395 1st Qu.: 728517 1st Qu.: 311299
Median :445908 Median : 766409 Median : 445912 Median : 571694 Median :725497 Median :1308198 Median :1590182 Median : 748431 Median : 363094
Mean :443520 Mean : 774832 Mean : 456002 Mean : 588302 Mean :727769 Mean :1300039 Mean :1578574 Mean : 757485 Mean : 373550
3rd Qu.:664009 3rd Qu.: 789017 3rd Qu.: 517820 3rd Qu.: 663263 3rd Qu.:739876 3rd Qu.:1438212 3rd Qu.:1756559 3rd Qu.: 773541 3rd Qu.: 420458
Max. :896504 Max. :1430309 Max. :2688042 Max. :3487913 Max. :911656 Max. :2107301 Max. :2513021 Max. :1430571 Max. :2591438
SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A BUV737-A BUV805-A Pacific Blue-A BV480-A BV570-A
Min. : 149874 Min. :1488 Min. :1049 Min. : 900.2 Min. :1349 Min. :1589 Min. :1413 Min. :1066 Min. :1369 Min. : 931.4
1st Qu.: 390511 1st Qu.:2558 1st Qu.:2735 1st Qu.:1898.4 1st Qu.:1965 1st Qu.:2781 1st Qu.:2155 1st Qu.:2113 1st Qu.:2001 1st Qu.:1910.5
Median : 456485 Median :2926 Median :3001 Median :2031.4 Median :2058 Median :3027 Median :3442 Median :2199 Median :2115 Median :2062.2
Mean : 471655 Mean :2848 Mean :2969 Mean :2057.7 Mean :2069 Mean :2922 Mean :2982 Mean :2196 Mean :2162 Mean :2066.5
3rd Qu.: 528192 3rd Qu.:3177 3rd Qu.:3229 3rd Qu.:2178.2 3rd Qu.:2154 3rd Qu.:3181 3rd Qu.:3525 3rd Qu.:2284 3rd Qu.:2241 3rd Qu.:2210.3

Max. :3461185 Max. :3729 Max. :4049 Max. :3471.2 Max. :3805 Max. :3574 Max. :3786 Max. :3612 Max. :7532 Max. :3673.6
BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor 488-A Spark Blue 550-A Spark Blue 574-A RB613-A RB705-A
Min. :1287 Min. :1185 Min. :1201 Min. :1531 Min. :1234 Min. : 831.2 Min. :3232 Min. :2996 Min. :1209 Min. :1566
1st Qu.:3201 1st Qu.:2026 1st Qu.:1886 1st Qu.:2127 1st Qu.:1869 1st Qu.:2076.1 1st Qu.:3430 1st Qu.:3162 1st Qu.:1949 1st Qu.:3269
Median :3413 Median :2172 Median :2065 Median :2284 Median :2064 Median :2171.9 Median :3495 Median :3224 Median :2140 Median :3576
Mean :3361 Mean :2563 Mean :2058 Mean :2287 Mean :2069 Mean :2188.4 Mean :3491 Mean :3219 Mean :2238 Mean :3329
3rd Qu.:3578 3rd Qu.:3553 3rd Qu.:2227 3rd Qu.:2439 3rd Qu.:2265 3rd Qu.:2281.4 3rd Qu.:3555 3rd Qu.:3278 3rd Qu.:2382 3rd Qu.:3705
Max. :4025 Max. :4044 Max. :3571 Max. :3187 Max. :3631 Max. :3333.9 Max. :3732 Max. :3477 Max. :3877 Max. :4085
RB780-A PE-A PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A PE-Fire 744-A PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A
Min. :1270 Min. : 771.5 Min. :1451 Min. :1635 Min. :1695 Min. :1382 Min. :1559 Min. :1097 Min. :1333 Min. :1468
1st Qu.:1948 1st Qu.:2265.0 1st Qu.:2098 1st Qu.:2074 1st Qu.:2688 1st Qu.:1969 1st Qu.:2061 1st Qu.:2042 1st Qu.:1928 1st Qu.:2024
Median :2080 Median :2498.5 Median :2225 Median :2177 Median :2890 Median :2076 Median :2188 Median :2180 Median :2065 Median :2129
Mean :2086 Mean :2473.1 Mean :2226 Mean :2292 Mean :2836 Mean :2121 Mean :2220 Mean :2188 Mean :2059 Mean :2135
3rd Qu.:2217 3rd Qu.:2695.7 3rd Qu.:2354 3rd Qu.:2409 3rd Qu.:3044 3rd Qu.:2185 3rd Qu.:2334 3rd Qu.:2317 3rd Qu.:2195 3rd Qu.:2237
Max. :3613 Max. :3391.4 Max. :3762 Max. :3661 Max. :3532 Max. :3894 Max. :4261 Max. :3838 Max. :3086 Max. :3229
Zombie NIR-A APC-Fire 750-A APC-Fire 810-A AF-A
Min. :1808 Min. :1079 Min. :1692 Min. : 648.6
1st Qu.:2179 1st Qu.:3171 1st Qu.:2966 1st Qu.:2792.7
Median :2302 Median :3266 Median :3151 Median :2878.0
Mean :2303 Mean :3224 Mean :3042 Mean :2846.6
3rd Qu.:2427 3rd Qu.:3339 3rd Qu.:3250 3rd Qu.:2938.4
Max. :2808 Max. :3609 Max. :3594 Max. :3389.5

Looking at the distribution of the values for individual fluorophores, everything seems rather suspiciously in the same linear-style range to each other than what we would normally anticipate for spectral flow cytometry data.

We recall back to Week 07 that unlike many commerical softwares, transformations in R are applied directly to the underlying values. When we ran gs_pop_get_data, we did not specify

that this transformation should be reversed, so we ended up retrieving the transformed data values.

We can correct this by setting the "inverse.transform" argument to "TRUE" within gs_pop_get_data(). After re-running the function, we get back

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#'
Downsampling <- function(x, subset){
  EventsInTheGate <- gs_pop_get_data(x, subset, inverse.transform=TRUE)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  summary(MeasurementData)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

```
  Time             SSC-W             SSC-H             SSC-A             FSC-
W             FSC-H             FSC-A            SSC-B-W           SSC-B-H

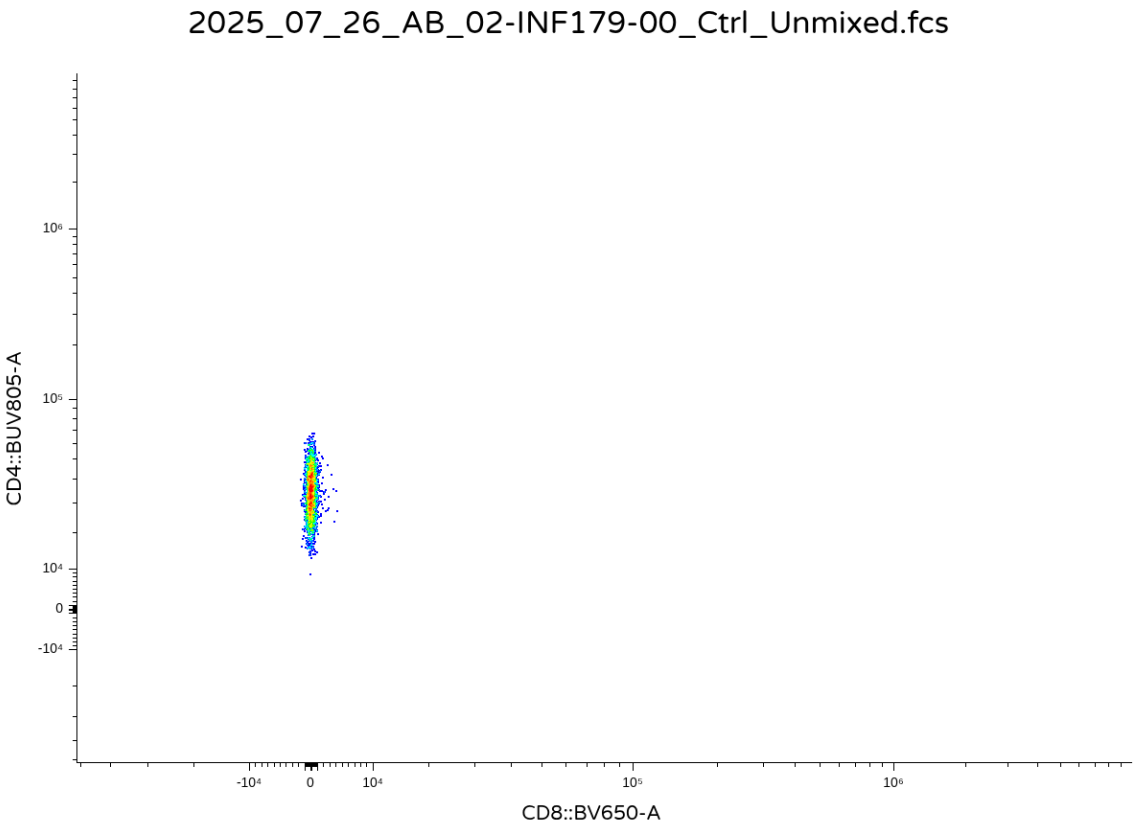
Min. : 19 Min. : 650411 Min. : 99153 Min. : 149656 Min. :617511 Min. : 775895 Min. :
940884 Min. : 626911 Min. : 104214
1st Qu.:223170 1st Qu.: 746861 1st Qu.: 376063 1st Qu.: 484396 1st Qu.:712088 1st
Qu.:1161711 1st Qu.:1400395 1st Qu.: 728517 1st Qu.: 311299
Median :445908 Median : 766409 Median : 445912 Median : 571694 Median :725497 Median
:1308198 Median :1590182 Median : 748431 Median : 363094
Mean :443520 Mean : 774832 Mean : 456002 Mean : 588302 Mean :727769 Mean :1300039
Mean :1578574 Mean : 757485 Mean : 373550
3rd Qu.:664009 3rd Qu.: 789017 3rd Qu.: 517820 3rd Qu.: 663263 3rd Qu.:739876 3rd
Qu.:1438212 3rd Qu.:1756559 3rd Qu.: 773541 3rd Qu.: 420458
Max. :896504 Max. :1430309 Max. :2688042 Max. :3487913 Max. :911656 Max. :2107301
Max. :2513021 Max. :1430571 Max. :2591438
SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A BUV737-A BUV805-A Pacific Blue-A
Min. : 149874 Min. :-2119 Min. : -6924 Min. :-10901.65 Min. :-3033.63 Min. :-1613 Min.
:-2572.1 Min. :-6585.3
1st Qu.: 390511 1st Qu.: 1856 1st Qu.: 2938 1st Qu.: -471.83 1st Qu.: -258.53 1st Qu.: 3311
1st Qu.: 335.6 1st Qu.: 203.2
Median : 456485 Median : 4887 Median : 6041 Median : -51.77 Median : 31.72 Median :
6526 Median :24648.6 Median : 477.3
```

Mean : 471655 Mean : 7112 Mean : 9492 Mean : 190.12 Mean : 314.42 Mean : 7416 Mean :19845.4 Mean : 508.5
3rd Qu.: 528192 3rd Qu.:10275 3rd Qu.: 12131 3rd Qu.: 409.62 3rd Qu.: 332.67 3rd Qu.:10415 3rd Qu.:32927.7 3rd Qu.: 755.7
Max. :3461185 Max. :68245 Max. :221842 Max. : 27270.46 Max. :90016.48 Max. :39197 Max. :84099.4 Max. :44859.2
BV480-A BV570-A BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor 488-A
Min. : -2878.1 Min. :-9883.27 Min. : -3565 Min. : -4697.76 Min. :-4487.03 Min. :-1888.3 Min. :-4111.13 Min. :-13602.64
1st Qu.: -148.2 1st Qu.: -432.71 1st Qu.: 11097 1st Qu.: -69.55 1st Qu.: -511.18 1st Qu.: 247.7 1st Qu.: -567.39 1st Qu.: 87.62
Median : 210.8 Median : 44.24 Median : 22284 Median : 388.91 Median : 52.05 Median : 754.8 Median : 51.38 Median : 389.27
Mean : 1620.6 Mean : 135.69 Mean : 29136 Mean : 21920.07 Mean : 130.00 Mean : 872.5 Mean : 132.75 Mean : 503.87
3rd Qu.: 612.8 3rd Qu.: 512.64 3rd Qu.: 39745 3rd Qu.: 36322.99 3rd Qu.: 565.60 3rd Qu.: 1324.0 3rd Qu.: 692.11 3rd Qu.: 747.52
Max. :3641199.2 Max. :55880.32 Max. :202829 Max. :217840.59 Max. :38725.14 Max. :10592.2 Max. :47909.14 Max. : 17081.65
Spark Blue 550-A Spark Blue 574-A RB613-A RB705-A RB780-A PE-A PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A
Min. :12220 Min. : 5962 Min. : -4391.1 Min. : -1719 Min. :-3726.5 Min. :-16555.3 Min. :-2327.7 Min. :-1413.16 Min. :-1175
1st Qu.:23663 1st Qu.: 9809 1st Qu.: -308.8 1st Qu.: 13788 1st Qu.: -314.5 1st Qu.: 692.4 1st Qu.: 155.6 1st Qu.: 80.22 1st Qu.: 2607
Median :29622 Median :11941 Median : 287.3 Median : 39412 Median : 100.4 Median : 1575.8 Median : 560.1 Median : 405.26 Median : 4434
Mean :30892 Mean :12259 Mean : 2005.0 Mean : 41475 Mean : 154.4 Mean : 1838.1 Mean : 614.3 Mean : 1320.70 Mean : 5100
3rd Qu.:36548 3rd Qu.:14184 3rd Qu.: 1104.3 3rd Qu.: 62495 3rd Qu.: 534.9 3rd Qu.: 2655.6 3rd Qu.: 1001.1 3rd Qu.: 1205.39 3rd Qu.: 6848
Max. :68934 Max. :27834 Max. :117274.7 Max. :253578 Max. :44938.6 Max. : 20731.3 Max. :76953.5 Max. :53463.78 Max. :33774
PE-Fire 744-A PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A Zombie NIR-A APC-Fire 750-A APC-Fire 810-A Min. : -2785.91 Min. : -1751.47 Min. : -6016.86 Min. :-3156.95 Min. :-2228.60 Min. :-771.6 Min. :-6337 Min. :-1188
1st Qu.: -245.89 1st Qu.: 40.83 1st Qu.: -17.61 1st Qu.: -376.49 1st Qu.: -74.91 1st Qu.: 412.5 1st Qu.:10074 1st Qu.: 5480
Median : 88.08 Median : 440.31 Median : 414.00 Median : 52.88 Median : 253.73 Median : 819.3 Median :13652 Median : 9477
Mean : 810.01 Mean : 1123.58 Mean : 646.52 Mean : 35.59 Mean : 297.63 Mean : 880.4 Mean :13846 Mean : 9351
3rd Qu.: 432.54 3rd Qu.: 931.15 3rd Qu.: 868.62 3rd Qu.: 463.24 3rd Qu.: 600.46 3rd

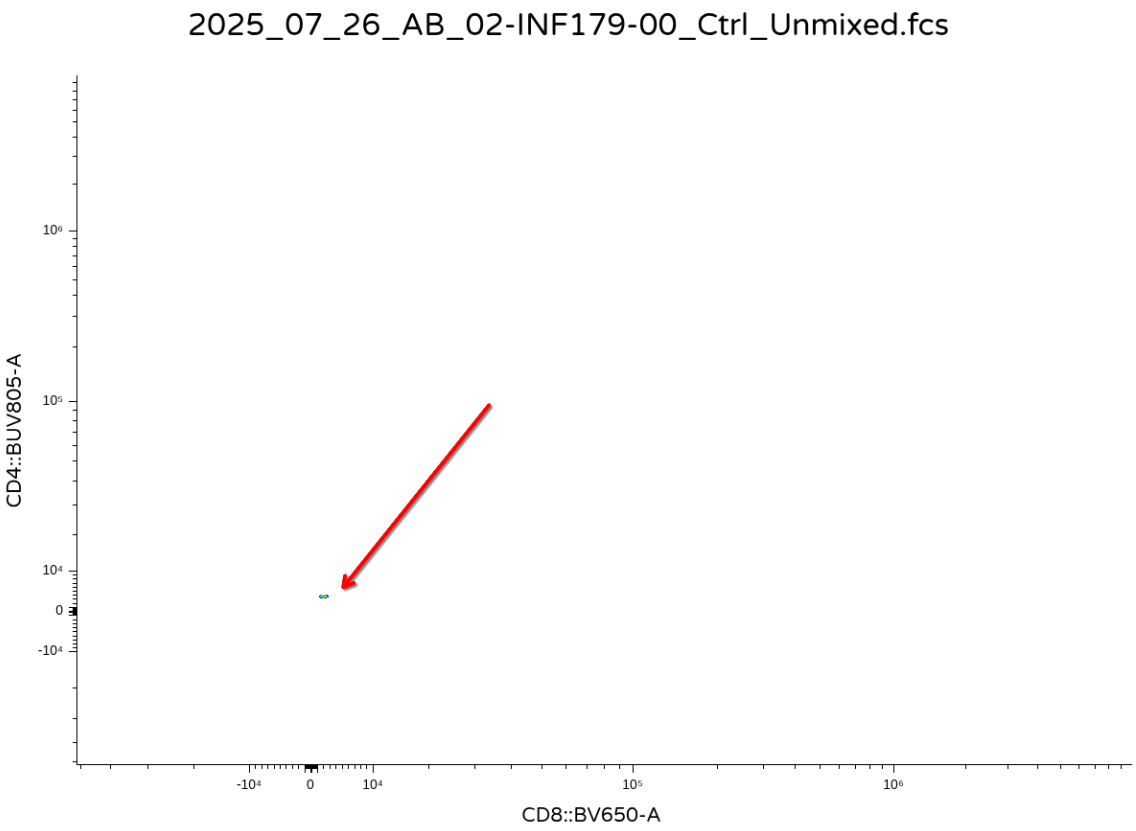
Qu.:1276.0 3rd Qu.:17382 3rd Qu.:12947
Max. :125136.12 Max. :426086.69 Max. :101602.37 Max. : 7755.29 Max. :12130.63 Max. :3554.9 Max. :44324 Max. :42018
AF-A
Min. :-25114
1st Qu.: 3416
Median : 4289
Mean : 4216
3rd Qu.: 5064
Max. : 20601

These results are more in-line with the usual value spread we would typically associate with unmixed spectral flow cytometry data for our respective fluorophores. So a win for paying attention. But what impact would it have had if we had retained the transformed values?

It would likely depend on what you are trying to do. If you plan to remain in R for your data analysis, then keeping these values transformed might make sense for a downstream analysis. Vice versa, if you are exporting out the data as new .fcs files, it is likely you or someone else might want to open them in commercial software. And instead of getting back something that looks like this for a downsampled CD4+ T cell population:



You will end up getting back a visual that looks like this



Since the commercial software applies scaling/transformation on top of the existing values (which were previously transformed in R). Consequently, lets go ahead and set 'inverse.transform' as Downsampling()'s third argument, but set the default equal to TRUE, since the main use case for today is ability to export out as .fcs files.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#'
Downsampling <- function(x, subset, inverse.transform=TRUE){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
```

```
  summary(MeasurementData)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

[[1]] Time SSC-W SSC-H SSC-A FSC-W FSC-H FSC-A SSC-B-W SSC-B-H
Min. : 19 Min. : 650411 Min. : 99153 Min. : 149656 Min. :617511 Min. : 775895 Min. : 940884 Min. : 626911 Min. : 104214
1st Qu.:223170 1st Qu.: 746861 1st Qu.: 376063 1st Qu.: 484396 1st Qu.:712088 1st Qu.:1161711 1st Qu.:1400395 1st Qu.: 728517 1st Qu.: 311299
Median :445908 Median : 766409 Median : 445912 Median : 571694 Median :725497 Median :1308198 Median :1590182 Median : 748431 Median : 363094
Mean :443520 Mean : 774832 Mean : 456002 Mean : 588302 Mean :727769 Mean :1300039 Mean :1578574 Mean : 757485 Mean : 373550
3rd Qu.:664009 3rd Qu.: 789017 3rd Qu.: 517820 3rd Qu.: 663263 3rd Qu.:739876 3rd Qu.:1438212 3rd Qu.:1756559 3rd Qu.: 773541 3rd Qu.: 420458
Max. :896504 Max. :1430309 Max. :2688042 Max. :3487913 Max. :911656 Max. :2107301 Max. :2513021 Max. :1430571 Max. :2591438
SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A BUV737-A BUV805-A Pacific Blue-A
Min. : 149874 Min. :-2119 Min. : -6924 Min. :-10901.65 Min. :-3033.63 Min. :-1613 Min. :-2572.1 Min. :-6585.3
1st Qu.: 390511 1st Qu.: 1856 1st Qu.: 2938 1st Qu.: -471.83 1st Qu.: -258.53 1st Qu.: 3311 1st Qu.: 335.6 1st Qu.: 203.2
Median : 456485 Median : 4887 Median : 6041 Median : -51.77 Median : 31.72 Median : 6526 Median :24648.6 Median : 477.3
Mean : 471655 Mean : 7112 Mean : 9492 Mean : 190.12 Mean : 314.42 Mean : 7416 Mean :19845.4 Mean : 508.5
3rd Qu.: 528192 3rd Qu.:10275 3rd Qu.: 12131 3rd Qu.: 409.62 3rd Qu.: 332.67 3rd Qu.:10415 3rd Qu.:32927.7 3rd Qu.: 755.7
Max. :3461185 Max. :68245 Max. :221842 Max. : 27270.46 Max. :90016.48 Max. :39197 Max. :84099.4 Max. :44859.2
BV480-A BV570-A BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor 488-A
Min. : -2878.1 Min. :-9883.27 Min. : -3565 Min. : -4697.76 Min. :-4487.03 Min. :-1888.3 Min. :-4111.13 Min. :-13602.64
1st Qu.: -148.2 1st Qu.: -432.71 1st Qu.: 11097 1st Qu.: -69.55 1st Qu.: -511.18 1st Qu.: 247.7 1st Qu.: -567.39 1st Qu.: 87.62
Median : 210.8 Median : 44.24 Median : 22284 Median : 388.91 Median : 52.05 Median : 754.8 Median : 51.38 Median : 389.27
Mean : 1620.6 Mean : 135.69 Mean : 29136 Mean : 21920.07 Mean : 130.00 Mean : 872.5 Mean : 132.75 Mean : 503.87
3rd Qu.: 612.8 3rd Qu.: 512.64 3rd Qu.: 39745 3rd Qu.: 36322.99 3rd Qu.: 565.60 3rd Qu.: 1324.0 3rd Qu.: 692.11 3rd Qu.: 747.52

Max. :3641199.2 Max. :55880.32 Max. :202829 Max. :217840.59 Max. :38725.14 Max. :10592.2 Max. :47909.14 Max. : 17081.65
Spark Blue 550-A Spark Blue 574-A RB613-A RB705-A RB780-A PE-A PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A
Min. :12220 Min. : 5962 Min. : -4391.1 Min. : -1719 Min. :-3726.5 Min. :-16555.3 Min. :-2327.7 Min. :-1413.16 Min. :-1175
1st Qu.:23663 1st Qu.: 9809 1st Qu.: -308.8 1st Qu.: 13788 1st Qu.: -314.5 1st Qu.: 692.4 1st Qu.: 155.6 1st Qu.: 80.22 1st Qu.: 2607
Median :29622 Median :11941 Median : 287.3 Median : 39412 Median : 100.4 Median : 1575.8 Median : 560.1 Median : 405.26 Median : 4434
Mean :30892 Mean :12259 Mean : 2005.0 Mean : 41475 Mean : 154.4 Mean : 1838.1 Mean : 614.3 Mean : 1320.70 Mean : 5100
3rd Qu.:36548 3rd Qu.:14184 3rd Qu.: 1104.3 3rd Qu.: 62495 3rd Qu.: 534.9 3rd Qu.: 2655.6 3rd Qu.: 1001.1 3rd Qu.: 1205.39 3rd Qu.: 6848
Max. :68934 Max. :27834 Max. :117274.7 Max. :253578 Max. :44938.6 Max. : 20731.3 Max. :76953.5 Max. :53463.78 Max. :33774
PE-Fire 744-A PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A Zombie NIR-A APC-Fire 750-A APC-Fire 810-A Min. : -2785.91 Min. : -1751.47 Min. : -6016.86 Min. :-3156.95 Min. :-2228.60 Min. :-771.6 Min. :-6337 Min. :-1188
1st Qu.: -245.89 1st Qu.: 40.83 1st Qu.: -17.61 1st Qu.: -376.49 1st Qu.: -74.91 1st Qu.: 412.5 1st Qu.:10074 1st Qu.: 5480
Median : 88.08 Median : 440.31 Median : 414.00 Median : 52.88 Median : 253.73 Median : 819.3 Median :13652 Median : 9477
Mean : 810.01 Mean : 1123.58 Mean : 646.52 Mean : 35.59 Mean : 297.63 Mean : 880.4 Mean :13846 Mean : 9351
3rd Qu.: 432.54 3rd Qu.: 931.15 3rd Qu.: 868.62 3rd Qu.: 463.24 3rd Qu.: 600.46 3rd Qu.:1276.0 3rd Qu.:17382 3rd Qu.:12947
Max. :125136.12 Max. :426086.69 Max. :101602.37 Max. : 7755.29 Max. :12130.63 Max. :3554.9 Max. :44324 Max. :42018
AF-A
Min. :-25114
1st Qu.: 3416
Median : 4289
Mean : 4216
3rd Qu.: 5064
Max. : 20601

Subsetting Now that Downsampling() is returning the correct underlying MFI measurement data for our gate of interest, let's start setting up the code to take the existing number of rows and downsample them to match a desired number of cells. A useful place to pick up at is confirming what type of object we are working with by adding class() back in on our last named object inside the function.

::: {.cell}

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#'
Downsampling <- function(x, subset, inverse.transform=TRUE){
 EventsInTheGate <- gs_pop_get_data(x, subset,
  inverse.transform=inverse.transform)
 MeasurementData <- exprs(EventsInTheGate[[1]])
 class(MeasurementData)
}
```

:::

::: {.cell}

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

:::

[[1]] [1] “matrix” “array”

While matrices are also rectangular in shape, they are often not as easy to manipulate compared to “data.frame” or “tibble” objects. Let’s go ahead and convert our matrix into a data.frame using as.data.frame().

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#'
Downsampling <- function(x, subset, inverse.transform=TRUE){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
```

```
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)
  class(MeasurementDataFramed)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

[[1]] [1] "data.frame"

Having a 'data.frame' object returned, lets switch out our current class() readout for nrow(), so that we can see how many cells we are working with before setting up the downsample code.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#'
Downsampling <- function(x, subset, inverse.transform=TRUE){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)
  nrow(MeasurementDataFramed)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling)
```

[[1]] [1] 9005

When we downsample from our existing data.frame, we would want to be able to retrieve out a specific number of cells (corresponding to individual rows) for a given specimen. These would ideally be selected randomly (and without replacement) so the return we get back would ideally be representative of what we would had seen for the original specimen.

Fortunately, dplyr's slice_sample() function is set up to do this for us, so we can set up its line of code within Downsampling(). In this case, we are passing slice_sample() our data.frame,

an outside argument (DownsampleCount), and setting the replace argument to FALSE to accomplish this.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population.
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  nrow(Downsampled_DataFrame)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling,
DownsampleCount=1000)
```

[[1]] [1] 1000

Seing as we have now downsampled to our target number of cells, let's switch from using nrow() for return(), so that our returned object is the actual MFI values. To keep things orderly for the website, lets temporarily switch our "DownsampleCount" argument to 10.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
```

```
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population.
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  return(Downsampled_DataFrame)
}
```

```
map(.x=SFC_GatingSet[1], subset="Tcells", .f=Downsampling,
DownsampleCount=10)
```

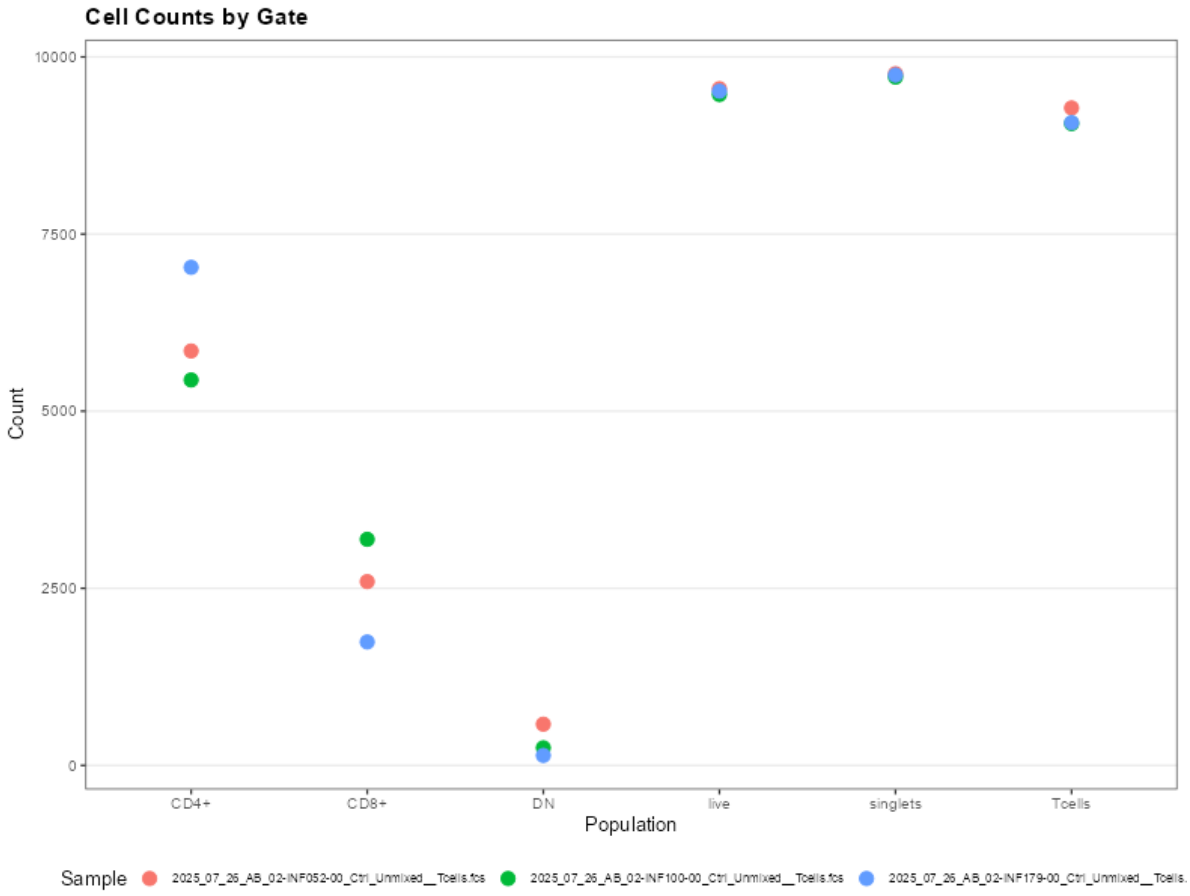
[[1]] Time SSC-W SSC-H SSC-A FSC-W FSC-H FSC-A SSC-B-W SSC-B-H SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A BUV737-A BUV805-A 1 625190 778347.9 477405 619312.0 772089.2 1305376 1679778 766872.4 349349 446510.2 18143.250 800.1056 1227.3851 7.162624 6731.030 22664.9141 2 575810 755777.8 284237 358033.3 705978.8 1261547 1484376 741820.2 219149 270948.6 5264.039 106.2783 175.6296 169.632782 14149.477 36511.6719 3 93311 763354.7 492748 626902.5 730219.4 1373134 1671148 739519.9 352503 434471.6 2793.462 9031.4492 206.7556 -525.446289 3315.611 35022.6328 4 98917 798562.1 299406 398490.5 736729.8 1064384 1306939 776499.8 269919 349320.1 7341.140 15915.9609 553.8736 -421.011566 5758.278 -253.3772 5 261580 775183.9 516092 666777.0 760414.6 1462900 1854018 754111.6 382687 480981.2 4214.797 2497.9021 -102.4402 -214.502548 5732.277 184.2956 6 136992 749347.7 292468 365267.0 720242.1 943009 1131991 742035.1 281622 348289.0 10935.342 6080.7583 466.6136 1046.610718 9306.068 20975.3398 7 161059 853481.3 391626 557075.8 757165.1 1342157 1693724 851406.2 319709 453670.4 1398.598 23950.0859 239.6974 321.851501 9513.285 25256.1895 8 888067 824867.4 569939 783540.2 751620.5 1309257 1640107 817643.3 425461 579792.2 11463.314 11668.8291 126.0130 48.929955 4749.728 25319.2793 9 102664 775890.8 538255 696045.2 735344.9 1231570 1509381 769444.6 366074 469456.1 5611.585 2525.5591 -2005.0074 -181.856873 4546.451 -472.6853 10 552359 751470.1 418597 524271.9 723971.3 1386510 1672989 741395.7 321409 397152.1 7767.877 5011.7354 -430.4062 455.375793 7583.418 37298.1367 Pacific Blue-A BV480-A BV570-A

BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor 488-A Spark Blue 550-A Spark Blue 574-A RB613-A 1 332.41650 -47.01189 -242.63307 16550.893 338.7156 83.73573 23.85567 1117.21008 582.94958 28543.12 11928.233 498.9186 2 151.78409 200.45068 172.84520 9363.221 2631.0618 -250.12549 497.96964 1404.52356 263.22577 28445.12 10249.930 -631.3950 3 430.37689 10.91681 165.43263 18228.156 -174.4520 279.74014 1637.08313 451.75687 -186.24971 25092.51 6995.734 -199.9772 4 57.61217 318.27567 -545.16650 74835.211 53979.5117 -828.32812 1737.69934 -1602.37830 870.52832 31934.47 12756.413 322.1815 5 193.89822 -19.46144 309.85352 30873.818 87663.4453 1231.11816 -93.13383 -93.52295 -46.13445 40405.02 15108.957 946.5299 6 586.39062 -60.85189 -554.41858 37862.176 -374.8628 -504.33298 927.51862 -443.79581 551.08972 33534.32 11985.690 -345.4969 7 576.56299 158.58598 739.37445 20361.158 699.5674 296.80823 901.38519 479.85083 625.94379 26687.98 15087.406 -30.3857 8 377.71939 -183.04483 -86.36446 12019.869 272.2732 -159.81361 -123.82663 1633.97900 151.99660 32715.76 7777.927 981.2333 9 1076.29993 265.54956 1241.17737 117128.297 81981.0000 643.58942 -537.41174 1885.70496 266.71893 29659.27 12103.782 486.8053 10 613.84637 -331.07144 284.85919 43782.598 538.8544 264.15646 2095.33691 890.09924 615.48645 23197.31 10494.148 864.8149 RB705-A RB780-A PE-A PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A PE-Fire 744-A PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A Zombie NIR-A 1 44838.508 -141.585068 1068.2528 578.6277 -64.876808 4269.905 -202.18224 1048.51550 793.43475 371.38504 -613.87531 2043.3976 2 27483.652 598.196106 1058.2180 293.4682 165.636078 2071.323 568.53308 249.52765 333.54977 -587.59106 868.64899 -102.8294 3 42719.836 -8.468122 2283.2017 -224.1359 127.101288 2758.384 -569.53607 1182.81165 1095.60669 81.92484 -34.87518 1089.9259 4 45776.945 -815.360107 567.6264 1034.4785 1201.292236 2185.473 256.07065 203.22432 887.91266 178.59489 625.20990 1009.0977 5 6368.570 608.893005 2592.7739 944.8301 2856.818115 3225.984 -271.71750 1770.93811 1544.25122 -448.50470 442.44052 1125.8389 6 34369.695 643.594238 4197.4761 1500.0323 4.783360 5015.515 92.60738 504.29309 -359.98053 698.12653 941.68384 899.7871 7 28408.859 1386.730591 4264.9146 -597.1237 508.787720 7236.833 960.18005 143.71167 136.92210 118.64117 1368.51392 640.4598 8 58408.574 164.525543 3364.6177 554.7925 -375.413147 7178.272 523.82959 185.51820 -98.60514 132.37144 -59.86410 1598.0328 9 8827.783 4133.052734 399.1211 194.7530 3084.345947 4067.024 72.06303 5995.08447 1741.64331 -400.03513 307.41357 833.0505 10 55555.387 -246.124710 942.3546 1063.8234 5.414372 5490.021 185.81908 55.72056 560.93927 187.70859 99.84937 1160.2867 APC-Fire 750-A APC-Fire 810-A AF-A 1 16674.268 12961.646 5846.889 2 10726.335 8901.772 3642.393 3 14285.583 10966.334 4965.288 4 8933.297 12931.965 3294.394 5 17426.600 10220.262 5342.306 6 12976.565 12468.265 2928.806 7 16802.729 18170.705 4243.143 8 19433.396 13158.336 5663.485 9 20943.109 14394.367 4701.339 10 13995.416 9620.977 5281.644

Alternate Scenario 1

All-in-all, our Downsampling() function appears to be in working order. Before moving on to figuring how to convert the data.frame into an .fcs, lets consider a couple things.

As we saw, for our dataset, the counts were fairly similar across the board for our 'Tcells' gate.



But what would happen in a scenario where we provided a downsample count that was greater than the number of cells present in the specimen? Would that work? Or would we get back an error?

Lets check with the CD8+ gate, using a count of 2500 (which would not be enough for INF179). Lets also switch back our function readout within the function to nrow() for an easier visual summary check.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
```

```
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population.
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  nrow(Downsampled_DataFrame)
}
```

```
map(.x=SFC_GatingSet, subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] [1] 2500

[[2]] [1] 2500

[[3]] [1] 1713

Based on the returns, it appears the default behavior for slice_sample() in this scenario where not enough cells are present is just to return all cells that are currently present. Which is good, so one potential worry off our list.

Alternate Scenario 2

Alternatively, what if we wanted to retrieve a certain percentage of cells from within a gate for each individual specimen, rather than a fixed count?

While we could write this as an entirely separate function, the smarter way (reusing existing code) would be to set up a conditional.

One way to implement this using our existing arguments would be, if our 'DownsampleCount' argument is less than 1, this would correspond to the desired downsampling proportion that we would to subsample for the respective gated cell population.

So in practice, we would modify the function as follows, and update the documentation, as it is not an immediately obvious practice.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE, DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData, check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }


  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  nrow(Downsampled_DataFrame)
}
```

```
# 0.1 or 10% of each .fcs file

map(.x=SFC_GatingSet, subset="CD8+", .f=Downsampling,
DownsampleCount=0.1)
```

[[1]] [1] 257

[[2]] [1] 308

[[3]] [1] 171

And with that, the beating heart of our Downsampling function has now been properly set up.

returnType

We should probably start figuring out what object format we want our Downsampling() function to be returning.

Having generated a subsetted-exprs-matrix, we could reinsert it into the .fcs file, and write it out to a specific folder. This would allow us to access it for subsequent use later on. We would still however need to make a few additional adjustments to the .fcs file metadata, so that we properly document the changes that have been made (so we don't end up confusing these downsampled files with the original ones).

Alternately, after downsampling, we may just want to return out outputs directly to R for continued analysis. In this scenario, we might want back the data as a 'data.frame' object (which wouldn't have any associated metadata) or as a 'flowFrame' or 'cytoframe' object (which would have corresponding metadata).

Let's start with the main goal, our downsampled output as an .fcs file, and then add conditionals to allow for the return the other two options.

Creating new .fcs files

Remembering back to Week 03, .fcs files in R are made up of 3 slots in the S4 object, 'exprs' (which we have been manipulating today), 'parameters' (containing general fluorophore/marker panel info), and 'description/keyword' (all the other metadata).

So far, we have not changed anything in terms of the number of columns, so we shouldn't need to make any changes to parameters (yey!). But we would need to swap out the existing exprs matrix (corresponding to that of the original .fcs file) for our downsampled one. Similarly, good reproducibility practice means we should update the appropiate keywords so that the generated .fcs files are not the originals.

Lets continue by converting our 'data.frame' object back to the original 'matrix' type object, using the as.matrix() function.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting
      DownsampleCount used for downsampling
    }


  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)
  class(DownsampledMatrix)
}
```

```
map(.x=SFC_GatingSet, subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] [1] “matrix” “array”

[[2]] [1] “matrix” “array”

[[3]] [1] “matrix” “array”

As mentioned, we will need to break out and copy the pieces from the original .fcs file into the new .fcs. The easiest way to gain access to all this information is to switch over from a cytoframe (working via a pointer) to a flowFrame (loaded into RAM). This will allow us access to the flowCore helper functions (parameters(), exprs(), and keyword()) to access the corresponding slots.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting
      DownsampleCount used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
```

```
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  return(OriginalParameters)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] An object of class 'AnnotatedDataFrame' rowNames: $P1 $P2 … $P43 (43 total) varLabels: name desc … maxRange (5 total) varMetadata: labelDescription

Seeing as we now have access to the 'flowFrame' contents, we can now cobble together our "DownsampledMatrix" (corresponding to the new 'exprs' slot) with the contents of the original 'description' and 'parameters' slots.

With all the components gathered, creating a new .fcs file is as simple as handing them off to the new() function.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
```

```
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting
      DownsampleCount used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix,
  parameters=OriginalParameters, description=OriginalDescription)

  return(NewFCS)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] flowFrame object '2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs' with 2500 cells and 43 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P39 APC-R700-A CD107a 4194275 -111 4194275 $P40 Zombie NIR-A Viability 4194275 -111 4194275 $P41 APC-Fire 750-A CD27 4194275 -111 4194275 $P42 APC-Fire 810-A CCR7 4194275 -111 4194275 $P43 AF-A NA 4194275 -111 4194275 472 keywords are stored in the 'description' slot

We are now able to get back a standard flowFrame object. Looking at the readout output, we see that it automatically updated to reflect the new number of downsampled cells, while retaining the metadata from the original .fcs file.

Creating new .fcs files

Remembering back to Week 03, .fcs files in R are made up of 3 slots in the S4 object, 'exprs' (which we have been manipulating today), 'parameters' (containing general fluorophore/marker panel info), and 'description/keyword' (all the other metadata).

So far, we have not changed anything in terms of the number of columns, so we shouldn't need to make any changes to parameters (yey!). But we would need to swap out the existing exprs matrix (corresponding to that of the original .fcs file) for our downsampled one. Similarly, good reproducibility practice means we should update the appropiate keywords so that the generated .fcs files are not the originals.

Lets continue by converting our 'data.frame' object back to the original 'matrix' type object, using the as.matrix() function.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }


  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)
```

```
  class(DownsampledMatrix)
}
```

```
map(.x=SFC_GatingSet, subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] [1] “matrix” “array”

[[2]] [1] “matrix” “array”

[[3]] [1] “matrix” “array”

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
```

```
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  return(OriginalParameters)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] An object of class 'AnnotatedDataFrame' rowNames: $P1 $P2 … $P43 (43 total) varLabels: name desc … maxRange (5 total) varMetadata: labelDescription

Seeing as we now have access to the 'flowFrame' contents, we can now cobble together our "DownsampledMatrix" (corresponding to the new 'exprs' slot) with the contents of the original 'description' and 'parameters' slots.

With all the components gathered, creating a new .fcs file is as simple as handing them off to the new() function.

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#'
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
```

```
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix,
  parameters=OriginalParameters, description=OriginalDescription)

  return(NewFCS)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500)
```

[[1]] flowFrame object '2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs' with 2500 cells and 43 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P39 APC-R700-A CD107a 4194275 -111 4194275 $P40 Zombie NIR-A Viability 4194275 -111 4194275 $P41 APC-Fire 750-A CD27 4194275 -111 4194275 $P42 APC-Fire 810-A CCR7 4194275 -111 4194275 $P43 AF-A NA 4194275 -111 4194275 472 keywords are stored in the 'description' slot

We are now able to get back a standard flowFrame object. Looking at the readout output, we see that it automatically updated to reflect the new number of downsampled cells, while retaining the metadata from the original .fcs file.

Updating keywords

Before calling it good, and saving this flowFrame as an .fcs, lets back up a couple lines and update a few important keywords within the 'description' slot, so that we can tell our "down-sampled in R" .fcs file apart from the original .fcs file.

A simpler way of doing this is setting up another argument (which we will designate as 'addon'), that will append a character value between the specimens corresponding tubename and the ending .fcs. For this particular instrument manufacturer, changing the "GUID" keyword for this .fcs file makes sense (although the equivalent keyword may vary depending on other platforms).

```
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs in the GUID
#' keyword to tell the downsampled file apart from the original.
#'
Downsampling <- function(x, subset, inverse.transform=TRUE, DownsampleCount,
  addon){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData, check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
```

```
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName) #Swtiching out .fcs for Updated Name
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix, parameters=OriginalParameters,
  description=OriginalDescription)

  return(NewFCS)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8")
```

[[1]] flowFrame object '2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells_CD8.fcs' with 2500 cells and 43 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P39 APC-R700-A CD107a 4194275 -111 4194275 $P40 Zombie NIR-A Viability 4194275 -111 4194275 $P41 APC-Fire 750-A CD27 4194275 -111 4194275 $P42 APC-Fire 810-A CCR7 4194275 -111 4194275 $P43 AF-A NA 4194275 -111 4194275 472 keywords are stored in the 'description' slot

From the readout, we can see that we are now able to distinguish our file from the original based on at least this single keyword.

Export as .fcs

Lets work in how to export the 'flowFrame' out as a .fcs file, ideally to a designated storage location. Since our now updated GUID keyword already contains ".fcs" at the end, we don't need to add anything else to the new file name in order to specify the file type. We will just need to add 'StorageLocation' as another Downsampling() argument (updating the documentation accordingly), and then do some adjustments internally to generate a full file.path().

```r
#' This function downsamples from a designated gate to our desired number
#' of cells, returning as a new .fcs file
#'
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs in the GUID
#' keyword to tell the downsampled file apart from the original.
#' @param StorageLocation A file.path to the folder you want to
#' store the new downsampled fcs file to.
#'
Downsampling <- function(x, subset, inverse.transform=TRUE, DownsampleCount,
  addon, StorageLocation){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)
```

```
  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName)
  #Swtiching out .fcs for Updated Name via the sub function
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix, parameters=OriginalParameters,
  description=OriginalDescription)

  StoreFCSFileHere <- file.path(StorageLocation, UpdatedGUID)

  return(StoreFCSFileHere)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8",
 StorageLocation="C:/Users/Macarena/MacarenaLM/notes/
  week10-Downsampling and concatenation/data")
```

As we have encountered previously, if no value is provided to an argument, it returns back as an error. Consequently, adding a default option would make sense in this case. We can use the getwd() function to identify the file.path to the current working directory, which will be used as the standin in case we don't end up specifying a 'StorageLocation' file.path.

By setting the default argument value for 'StorageLocation' equal to NULL (i.e. nothing), we can use an 'if' conditional in combination with is.null() to handle this situation when encountered.

```
#' This function downsamples from a designated gate to our
#' desired number of cells, returning as a new .fcs file
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs
#' in the GUID keyword to tell the downsampled file apart from the original.
#' @param StorageLocation A file.path to the folder you want to store the
#' new downsampled fcs file to. Default NULL results in .fcs file being
```

```r
#' stored in current working directory
Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount, addon, StorageLocation=NULL){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData,
  check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName)
  #Swtiching out .fcs for Updated Name via the sub function
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix,
  parameters=OriginalParameters, description=OriginalDescription)

  if (is.null(StorageLocation)){StorageLocation <- getwd()}

  StoreFCSFileHere <- file.path(StorageLocation, UpdatedGUID)

  return(StoreFCSFileHere)
}
```

```
map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8")
```

[[1]] [1] "c:/Users/Macarena/MacarenaLM/2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells_CD8.fcs

Having the full file path and corresponding new name now specified within Downsampling(), we are now ready to write our first new .fcs file. This is accomplished through the flowCore packages write.FCS() function, which we will add in at the end of our function.

```
#' This function downsamples from a designated gate to our
#' desired number of cells, returning as a new .fcs file
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file,
#' default is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample
#' from each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs
#'  in the GUID keyword to tell the downsampled file
#' apart from the original.
#' @param StorageLocation A file.path to the folder you want
#' to store the new downsampled fcs file to. Default NULL results in .fcs
#' file being stored in current working directory

Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount, addon, StorageLocation=NULL){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData, check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }
```

```
  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName) #Swtiching out .fcs
  for Updated Name via the sub function
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix,
  parameters=OriginalParameters, description=OriginalDescription)

  if (is.null(StorageLocation)){StorageLocation <- getwd()}

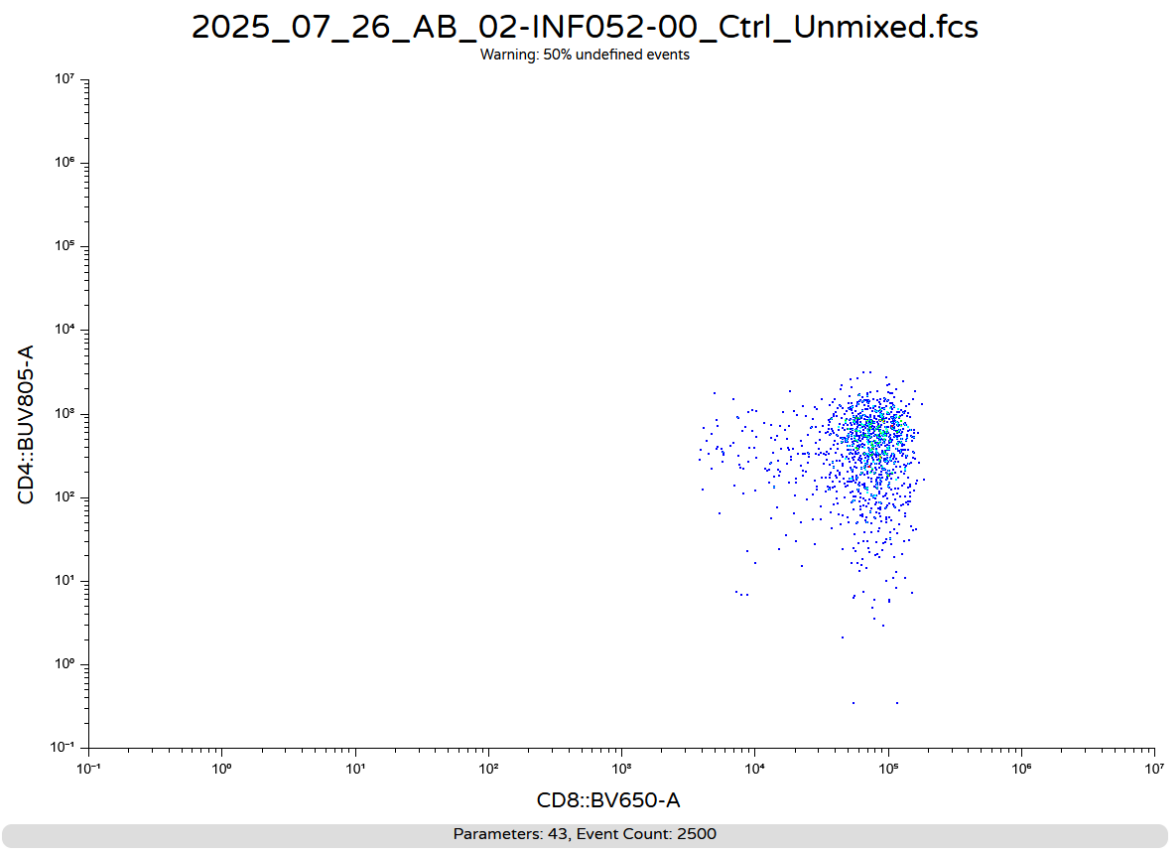
  StoreFCSFileHere <- file.path(StorageLocation, UpdatedGUID)
  write.FCS(NewFCS, filename = StoreFCSFileHere, delimiter="#")

  return(StoreFCSFileHere)
}
```

```
# Returning to Working directory, provide StorageLocation
# value if trying to store elsewhere

map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8")
```

[[1]] [1] “c:/Users/Macarena/MacarenaLM/2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells_CD8.fcs



Alternate export options

Now that we can export as a '.fcs' file, lets wrap up by providing the option to instead return either the 'data.frame' or the 'flowFrame' object (if we wished to continue working with them in R). We can do set up within Downsampling() a new argument (returnType) and a couple branching conditional statements with 'if' and 'ifelse' to designate the different outcomes for different provided argument values.

```
#' This function downsamples from a designated gate to our
#' desired number of cells, returning as a new .fcs file
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
```

```
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs in the GUID
#' keyword to tell the downsampled file apart from the original.
#' @param StorageLocation A file.path to the folder you want to
#' store the new downsampled fcs file to. Default NULL results in .fcs file
#' being stored in current working directory
#' @param returnType Whether to return as a "fcs" file (default),
#' or "flowFrame" or "data.frame"

Downsampling <- function(x, subset, inverse.transform=TRUE, DownsampleCount,
  addon, StorageLocation=NULL, returnType="fcs"){
  EventsInTheGate <- gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData, check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)

  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- parameters(flowFrame)
  OriginalDescription <- keyword(flowFrame)

  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName)
  #Swtiching out .fcs for Updated Name via the sub function
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix,
  parameters=OriginalParameters, description=OriginalDescription)
```

```
  if (is.null(StorageLocation)){StorageLocation <- getwd()}

  StoreFCSFileHere <- file.path(StorageLocation, UpdatedGUID)


  if (returnType == "fcs"){
    write.FCS(NewFCS, filename = StoreFCSFileHere, delimiter="#")
    # Write out .fcs file
  } else if (returnType == "data.frame"){
    return(Downsampled_DataFrame) #Return data.frame
    without metadata
  } else {
    return(NewFCS) #All other criterias return a flowFrame
    with metadata
    }

}
```

```
Data <- map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8",
  returnType="data.frame")
head(Data[[1]], 3)
```

Time SSC-W SSC-H SSC-A FSC-W FSC-H FSC-A SSC-B-W SSC-B-H SSC-B-A BUV395-A BUV563-A BUV615-A BUV661-A 1 460247 797842.8 552501 734681.5 722602.2 1543499 1858893 786553.5 415879 545185.1 -46.97797 21329.977 -579.57404 85.72366 2 700507 762130.3 582036 739312.1 713487.4 1240854 1475556 756714.6 399582 503949.2 679.22980 3045.998 -43.23803 -430.71301 3 414649 790553.6 457086 602251.6 717903.7 1222881 1463185 760655.6 407898 517116.5 4368.94385 22716.822 -236.07365 -419.55197 BUV737-A BUV805-A Pacific Blue-A BV480-A BV570-A BV605-A BV650-A BV711-A BV750-A BV786-A Alexa Fluor 488-A 1 -393.2001 95.80434 549.01611 -803.5530 -14.75448 6148.159 41542.86 -563.86432 2115.5513 -1631.4536 -299.22055 2 2001.5905 -119.28712 91.83144 -685.7737 -433.16922 19179.859 40806.29 46.78167 1033.9384 181.5461 -92.63408 3 8942.4414 -572.28735 -423.89954 224.2744 -843.47687 37724.812 81334.76 -12.54355 -75.0267 851.1154 929.99866 Spark Blue 550-A Spark Blue 574-A RB613-A RB705-A RB780-A PE-A PE-Dazzle594-A PE-Cy5-A PE-Fire 700-A PE-Fire 744-A 1 20607.09 10368.57 594.3114 423.9983 -346.9290 291.1394 789.98450 9697.097 1706.174 141.2359 2 22453.01 6847.99 -952.8335 22330.5977 129.5231 686.1887 -78.89325 2195.811 3959.893 147.0559 3 25610.48 11467.25 -403.1342 12677.1162 -513.3351 1343.1682 215.57680 2642.032 6109.715 189.3594 PE-Vio770-A APC-A Alexa Fluor 647-A APC-R700-A Zombie NIR-A APC-Fire 750-A APC-Fire 810-A AF-A 1 -207.6845 -410.0576 485.8525 877.6296 1060.079 5557.021 1133.32 6661.873 2 482.9545 381.9060

-217.6469 894.4412 296.634 11385.277 10244.24 6228.993 3 762.5884 868.0554 235.5182 409.6299 1039.831 14608.673 13669.77 4170.205

And there we go, Downsampling() now has the functionality to return objects in different formats, depending on what our use case may be.

Dependencies

One important thing before proceeding, Downsampling currently works because we called library() on all the packages needed to run the various functions we were using inside of it. If we were to close Positron and reopen, if we forgot to run one of these libraries (let's say dplyr), we would get the following style error:

```
Data <- map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8",
  returnType="data.frame")
head(Data[[1]], 3)
```

```
> Data <- map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling, DownsampleCount=2500, addon="CD8",
+   returnType="data.frame")
+ head(Data[[1]], 3)
Error in `map()`:
i In index: 1.
Caused by error in `slice_sample()`:
! could not find function "slice_sample"
▸ Show Traceback
>
```

Since dplyr is not attached to our local environment, when slice_sample() is encountered within the function, and not defined elsewhere, we get an error returned. While we could remember to load all our various R packages at the start to avoid this issue, this opens another can of worms, as many R packages have functions with identical names, which results in the last package called masking those with identical names before it. This can often cause functions to fail, so less than ideal.

There are two ways around this. Later on in the course, we will see how to use the @importFrom tag within a Roxygen2 skeleton, alongside the devtools package load_all() function to specify function-level dependencies from the get-go. However, since a couple additional setup steps are needed, for now, we will default to updating our functions to use the "packageName :: function name" option, with the '::' telling R to use the function from that package regardless if it is currently attached to the local environment or not.

In our case, I will go ahead and add the @importFrom tags (syntax is package name, followed by functions being imported from it) within the roxygen skeleton, and then within the function do the equivalent second option using '::'

```r
#' This function downsamples from a designated gate to our
#' desired number of cells, returning as a new .fcs file
#' @param x A GatingSet object, typically iterated in.
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs
#' in the GUID keyword to tell the downsampled file apart from the original.
#' @param StorageLocation A file.path to the folder you want
#' to store the new downsampled fcs file to. Default NULL results
#' in .fcs file being stored in current working directory
#' @param returnType Whether to return as a "fcs" file (default),
#' or "flowFrame" or "data.frame"
#' @importFrom flowWorkspace gs_pop_get_data
#' @importFrom flowCore parameters keyword write.FCS
#' @importFrom Biobase exprs
#' @importFrom dplyr slice_sample

Downsampling <- function(x, subset, inverse.transform=TRUE,
DownsampleCount, addon, StorageLocation=NULL, returnType="fcs"){
  EventsInTheGate <- flowWorkspace::gs_pop_get_data(x, subset,
   inverse.transform=inverse.transform)
  MeasurementData <- Biobase::exprs(EventsInTheGate[[1]])
  MeasurementDataFramed <- as.data.frame(MeasurementData, check.names = FALSE)

  if (DownsampleCount < 1) {
      Count <- nrow(EventsInTheGate) # Original Count
      Count <- as.numeric(Count) #Sanity Check on Value Type
      Count <- Count*DownsampleCount # Target Cells
      Count <- round(Count, 0)
      DownsampleCount <- Count # Over-writting DownsampleCount
      used for downsampling
    }

  Downsampled_DataFrame <- dplyr::slice_sample(MeasurementDataFramed,
  n = DownsampleCount, replace = FALSE)

  DownsampledMatrix <- as.matrix(Downsampled_DataFrame)
```

```
  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- flowCore::parameters(flowFrame)
  OriginalDescription <- flowCore::keyword(flowFrame)

  OriginalName <- OriginalDescription$`GUID`
  UpdatedName <- paste0("_", addon, ".fcs")
  UpdatedGUID <- sub(".fcs", UpdatedName, OriginalName)
  #Swtiching out .fcs for Updated Name via the sub function
  OriginalDescription$`GUID` <- UpdatedGUID

  NewFCS <- new("flowFrame", exprs=DownsampledMatrix, parameters=OriginalParameters,
  description=OriginalDescription)

  if (is.null(StorageLocation)){StorageLocation <- getwd()}

  StoreFCSFileHere <- file.path(StorageLocation, UpdatedGUID)

  if (returnType == "fcs"){
    flowCore::write.FCS(NewFCS, filename = StoreFCSFileHere,
    delimiter="#") # Write out .fcs file
  } else if (returnType == "data.frame"){
    return(Downsampled_DataFrame)
    #Return data.frame without metadata
  } else {
    return(NewFCS)
    #All other criterias return a flowFrame with metadata
    }

}
```

```
purrr::map(.x=SFC_GatingSet[1], subset="CD8+", .f=Downsampling,
DownsampleCount=2500, addon="CD8",
  returnType="flowFrame")
```

[[1]] flowFrame object '2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells_CD8.fcs' with 2500 cells and 43 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P39 APC-R700-A CD107a 4194275 -111 4194275 $P40 Zombie NIR-A Viability 4194275 -111 4194275 $P41 APC-Fire 750-A CD27 4194275 -111 4194275 $P42 APC-Fire 810-A CCR7 4194275 -111 4194275 $P43 AF-A NA 4194275 -111 4194275 472 keywords are stored in the 'description' slot

Now, with the flowWorkspace, flowCore, Biobase and dplyr dependencies specied within the function, Downsample runs without issues regardless of whether the packages are attached to our local environment.

At last

We can now remove the indexing ([1]) from SFC_GatingSet that we used throughout the building process, and proceed to downsample our entire GatingSet based on our parameters of interests.

For example, returning 2500 CD4+ T cells as .fcs files (save it on Data folder)

```
purrr::walk(.x=SFC_GatingSet, subset="CD4+", .f=Downsampling,
DownsampleCount=2500, addon="CD4",
  returnType="fcs", StorageLocation="C:/Users/Macarena/MacarenaLM/notes/
    week10-Downsampling and concatenation/data")
```

Or 10% of all T cells (save it on Data folder)

```
purrr::walk(.x=SFC_GatingSet, subset="Tcells", .f=Downsampling,
DownsampleCount=0.1, addon="_Tcells",
  returnType="fcs", StorageLocation="C:/Users/Macarena/MacarenaLM/notes/
    week10-Downsampling and concatenation/data")
```

Export

Now that we have fully assembled Downsampling(), there is actually another useful use case when it comes to its ability to export out as fully-compatible .fcs files. If instead of providing a downsample number, we provided an implausibly high number (lets say 10,000,000) we would basically directly export out the entire gated cell population of interest as its own .fcs file.

This can be quite useful, as you can set up your cleanup gates and major cell population gates (double checking to make sure they are applied correctly), and then export out your actual cell populations of interest as smaller .fcs files for future analysis. That way, if you are implementing hybrid workflows with large spectral flow cytometry files relying on commercial software (similar to the Flow-Jo/CytoML example from Week 05), you do not crash your RAM just trying to open the workspace.

```
AllDNs <- purrr::map(.x=SFC_GatingSet, subset="Tcells", .f=Downsampling,
DownsampleCount=10000000, addon="DN_All",
  returnType="fcs", StorageLocation="C:/Users/Macarena/MacarenaLM/notes/
    week10-Downsampling and concatenation/data")
```

It has created a new .fcs

Concatenate

Nested Functions So far, we have continued the process of learning how to write our own functions to carry out useful tasks, culminating in creating our Downsampling function. While downsampling itself can be useful in certain situations, it is often used as part of a workflow where the downsampled outputs are combined together into a single .fcs file (concatenation) for use in unsupervised workflows (like dimensionality visualization).

Unlike Downsampling(), which works within an individual .fcs files at a time, if we were to build out a Concatenate() function, we would need to first retrieve the downsampled returns from every .fcs file via iteration, then combine them together, before outputting them as a new .fcs file.

The tricky part is that in practice, often when we Concatenate, we want to add in keywords to be able to tell the cells coming from the individual contributors of the concatenated file apart after the fact. For .fcs files, these keywords end up becoming additional columns in the 'exprs' matrix, which causes the need to not only modify exprs, but also parameters and description slots rather extensively before we can create the new .fcs file.

However, I promised to not drown you in your first forray to the deep-end of the pool, so there will be no more function building in this session. I will provide the out code for Concatenate() and its various nested helper functions below, and instead focus on how to use it for actual implementation within our workflows for the rest of the time we have left.

If, however, you are not burnt out on function building and want to optionally read through the equivalent Concatenate walk-through, follow the link here (https://umgcccfcsr.github.io/CytometryInR/course/10_Downsampling/concatenate.html)

Concatenate code

Briefly as far as explainers go, Concatenate needs to iterate through various .fcs files to retrieve the Downsampling outputs, as well as extract corresponding metadata for each iterated .fcs file. These get added in as new columns, with the cascading changes ending up applied to 'exprs', 'parameters', and 'keyword' slots, before being outputted as our return object type of

choice. Because of all these moving parts, Concatenate is an example of a nested function, with several smaller helper functions that are called to help carry out the individual tasks.

To see the Concatenate and associated helper functions, click the code-show arrow below. To skip the code and start using them, run the "Run Cell" option to get them to appear correctly in your local environment.

```
#' Concatenate Internal
#'
#' @param x TBD
#' @param y TBD
#' @param metadata TBD
#'
#' @importFrom dplyr filter bind_cols
#'
KeywordAppend <- function(x, y, metadata) {
  df <- y
  rownames(metadata) <- NULL
  AddThisRow <- metadata |> filter(name %in% x)
  ExpandedData <- bind_cols(df, AddThisRow)
  return(ExpandedData)
}

#' Concatenate Internal
#'
#' @param DictionaryList TBD
#' @param data TBD
#'
#' @importFrom dplyr left_join select rename
#' @importFrom tidyselect all_of
#' @importFrom rlang sym
#'
KeywordTranslate <- function(DictionaryList, data) {

  for (Entry in DictionaryList) {
    ColumnName <- names(Entry)[1]
    KeyName <- names(Entry)[2]

    data <- data |> dplyr::left_join(Entry, by = ColumnName) |>
      dplyr::select(-tidyselect::all_of(ColumnName))
      |> dplyr::rename(!!ColumnName := !!rlang::sym(KeyName))
  }
```

```
  return(data)
}

#' Concatenate Internal
#'
#' @param x TBD
#' @param data TBD
#'
#' @importFrom dplyr select pull
#' @importFrom tidyselect all_of
#' @importFrom tibble tibble
#'
#'
ColumnToKeyword <- function(x, data){
  IndividualColumn <- data |> dplyr::select(tidyselect::all_of(x))

  if(!is.numeric(IndividualColumn)){ # Is not numeric
    Values <- IndividualColumn |> dplyr::pull(x) |> unique()

    Dictionary <- tibble::tibble(Values = Values,
    Values_Key = seq(1000, by = 1000, length.out = length(Values)))
    colnames(Dictionary) <- gsub("Values", x, colnames(Dictionary))
    return(Dictionary)
  } else { # Is numeric already
    Values <- IndividualColumn |> dplyr::pull(x) |> unique()
    Dictionary <- tibble::tibble(Values = Values,
    Values_Key = Values)
    colnames(Dictionary) <- gsub("Values", x,
    colnames(Dictionary))
    return(Dictionary)
  }
}

#' Concatenate Internal
#'
#' @param flowFrame TBD
#' @param NewColumns TBD
#'
#' @importFrom flowCore pData parameters
#'
ParameterUpdate <- function(flowFrame, NewColumns){
    NewColumnLength <- ncol(NewColumns)
```

```
    NewColumnNames <- colnames(NewColumns)
    OldParameters <- pData(parameters(flowFrame))
    NewParameter <- max(as.integer(gsub("\\$P", "",
    rownames(OldParameters)))) + 1
    NewParameter <- seq(NewParameter, length.out = NewColumnLength)
    NewParameter <- paste0("$P", NewParameter)

    UpdatedParameters <- do.call(rbind,
    lapply(NewColumnNames, function(i){
                        vec <- NewColumns[,i]
                        rg <- range(vec)
                        data.frame(name = i, desc = NA, range = diff(rg) + 1,
                        minRange = rg[1], maxRange = rg[2])
                    }))
    rownames(UpdatedParameters) <- NewParameter
    return(UpdatedParameters)
}

#' Concatenates together .fcs files present in the GatingSet on the
#'  basis of a given gate
#'
#' @param gs A GatingSet object
#' @param subset The gate from which to retrieve cell counts from
#' @param inverse.transform Whether to revert values back to their
#' original untransformed values before export as an .fcs file, default
#' is set to TRUE
#' @param DownsampleCount The desired number of cells to downsample from
#' each gated population. If value is less than 1, subsets out the
#' equivalent proportion from that specimen
#' @param addon An additional character value to add before .fcs
#' in the GUID keyword to tell the downsampled file apart from the original.
#' @param StorageLocation A file.path to the folder you want to store
#' the new downsampled fcs file to. Default NULL results in .fcs file being
#' stored in current working directory
#' @param returnType Whether to return as a "fcs" file (default),
#' or "flowFrame" or "data.frame"
#' @param desiredCols A vector containing the names of the columns
#' from the pData metadata that need to be added as keywords to the
#' concatenated .fcs file.
#' @param specimenIndex Which specimen in the GatingSet to use as
#' the metadata framework for the new fcs file. Default is set to 1.
#' @param filename Desired name for the concatenated file,
```

```
#' default is MyConcatenatedFCS
#' @importFrom flowCore pData parameters keyword exprs write.FCS
#' @importFrom flowWorkspace gs_pop_get_data
#' @importFrom dplyr select bind_rows
#' @importFrom tidyselect all_of
#' @importFrom purrr map map2 flatten
#'
Concatenate <- function(gs, subset, inverse.transform=TRUE, DownsampleCount,
  addon, StorageLocation=NULL, returnType="flowFrame", desiredCols,
  specimenIndex=1, filename="MyConcatenatedFCS"){

  Metadata <- flowCore::pData(gs)
  DesiredMetadata <- Metadata
  |> dplyr::select(tidyselect::all_of(desiredCols))

  dataFrameList <- purrr::map(.x=gs, subset=subset,
  .f=Downsampling, DownsampleCount=DownsampleCount,
  addon=addon, returnType="data.frame",
  inverse.transform=inverse.transform, StorageLocation=StorageLocation)

  TheFileNames <- DesiredMetadata |> dplyr::pull(name)

  ExpandedDataframes <- purrr::map2(.x=TheFileNames, .y=dataFrameList,
   .f=KeywordAppend, metadata=DesiredMetadata)

  CombinedData <- dplyr::bind_rows(ExpandedDataframes)

  NewData <- CombinedData |> dplyr::select(tidyselect::all_of(desiredCols))
  OldData <- CombinedData |> dplyr::select(!tidyselect::all_of(desiredCols))

  Dictionaries <- purrr::map(.x=desiredCols, .f=ColumnToKeyword, data=NewData)

  EventsInTheGate <- flowWorkspace::gs_pop_get_data(gs[[specimenIndex]],
  subset, inverse.transform=inverse.transform)
  flowFrame <- EventsInTheGate[[1, returnType = "flowFrame"]]
  OriginalParameters <- flowCore::parameters(flowFrame)
  OriginalDescription <- flowCore::keyword(flowFrame)

  NewKeywords <- purrr::flatten(Dictionaries)
  NewDescriptions <- c(OriginalDescription, NewKeywords)

  TranslatedNewData <- KeywordTranslate(data=NewData,
```

```
DictionaryList=Dictionaries)

NewDataMatrix <- as.matrix(TranslatedNewData)
OldDataMatrix <- as.matrix(OldData)

new_fcs <- new("flowFrame", exprs=OldDataMatrix,
parameters=OriginalParameters, description=NewDescriptions)

NewParameters <- ParameterUpdate(flowFrame=new_fcs,
NewColumns=NewDataMatrix)

pd <- pData(parameters(new_fcs))
pd <- rbind(pd, NewParameters)
new_fcs@exprs <- cbind(exprs(new_fcs), NewDataMatrix)
pData(parameters(new_fcs)) <- pd
new_pid <- rownames(pd)
new_kw <- new_fcs@description

for (i in new_pid){
  new_kw[paste0(i,"B")] <- new_kw["$P1B"]
  #Unclear Purpose
  new_kw[paste0(i,"E")] <- "0,0"
  new_kw[paste0(i,"N")] <- pd[[i,1]]
  #new_kw[paste0(i,"V")] <- new_kw["$P1V"]
  # Extra Unclear Purpose
  new_kw[paste0(i,"R")] <- pd[[i,5]]
  new_kw[paste0(i,"DISPLAY")] <- "LIN"
  new_kw[paste0(i,"TYPE")] <- "Identity"
  new_kw[paste0("flowCore_", i,"Rmax")] <- pd[[i,5]]
  new_kw[paste0("flowCore_", i,"Rmin")] <- pd[[i,4]]
}

UpdatedParameters <- parameters(new_fcs)
UpdatedExprs <- exprs(new_fcs)

UpdatedFCS <- new("flowFrame", exprs=UpdatedExprs,
parameters=UpdatedParameters, description=new_kw)

AssembledName <- paste0(filename, ".fcs")
UpdatedFCS@description$GUID <- AssembledName
UpdatedFCS@description$`$FIL` <- AssembledName
#UpdatedFCS@description$CREATOR <- "CytometryInR_2026"
```

```
  #UpdatedFCS@description$GROUPNAME <- filename
  #UpdatedFCS@description$TUBENAME <- filename
  #UpdatedFCS@description$USERSETTINGNAME <- filename
  #Date <- Sys.time()
  #Date <- as.Date(Date)
  #UpdatedFCS@description$`$DATE` <- Date

  if (is.null(StorageLocation)){StorageLocation <- getwd()}

  StoreFCSFileHere <- file.path(StorageLocation, AssembledName)

  if (returnType == "fcs"){
    flowCore::write.FCS(UpdatedFCS, filename = StoreFCSFileHere,
    delimiter="#") # Write out .fcs file
  } else if (returnType == "data.frame"){
    return(Downsampled_DataFrame)
    #Return data.frame without metadata
  } else {
    return(UpdatedFCS)
    #All other criterias return a flowFrame with metadata
    }
}
```

Update Metadata

With Concatenate and it's helpers now active as functions within your local environment, we can now focus on the workflow needed to run these on our GatingSet. The way Concatenate was set up, additional keywords can be added in by retrieving the corresponding columns from the 'GatingSet' metadata (which is visible via pData()).

```
CurrentMetadata <- pData(SFC_GatingSet)
CurrentMetadata
```

2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs

As you can see, in our current GatingSet, it's just the standard name column. To edit the GatingSet metadata, we can repeat the steps used back during Week 07 to merge in additional metadata that is stored for the respective specimens in a .csv file (located in our case within the Week 10 data folder).

```
TheCSV <- list.files("C:/Users/Macarena/MacarenaLM/notes/
  week10-Downsampling and concatenation/data", pattern=".csv",
  full.names=TRUE)
AdditionalMetadata <- read.csv(TheCSV, check.names=FALSE)
colnames(CurrentMetadata)
```

[1] "name"

```
colnames(AdditionalMetadata)
```

[1] "name" "condition" "infant_sex" "HEU_status"

Seeing as both our data.frames have a column in common (and the names present are equivalent for both), we can use dplyr packages left_join() function to combine both data.frames together them. Once this is accomplished, we can then assign this back to our GatingSet.

```
UpdatedMetadata <- left_join(CurrentMetadata,
AdditionalMetadata, by="name")
rownames(UpdatedMetadata) <- UpdatedMetadata$name
pData(SFC_GatingSet) <- UpdatedMetadata
pData(SFC_GatingSet)
```

2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs Ctrl Male 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs Ctrl Female 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs Ctrl Male HEU_status 2025_07_26_AB_02-INF052-00_Ctrl_Unmixed___Tcells.fcs HEU-hi 2025_07_26_AB_02-INF100-00_Ctrl_Unmixed___Tcells.fcs HEU-lo 2025_07_26_AB_02-INF179-00_Ctrl_Unmixed___Tcells.fcs HU

Our metadata is now assembled, and we can provide the columns we want to add in as keywords to Concatenate()'s 'desiredCols' argument in the form of a vector (ex. desiredCols=c("name", "condition", "infant_sex", "HEU_status"))

Concatenate

Now that we have our functions, updated metadata, and our assembled GatingSet object, we can concatenate together gates of interest. In the example below, we are downsampling for 2000 CD4+ T cells, appending four keyword columns, and having R return the values as a flowFrame.

```
Concatenate(gs=SFC_GatingSet, subset="CD4+", addon="CD4",
DownsampleCount=2000, desiredCols=c("name", "condition", "infant_sex", "HEU_status"),
returnType="flowframe", filename="ConcatenatedCD4")
```

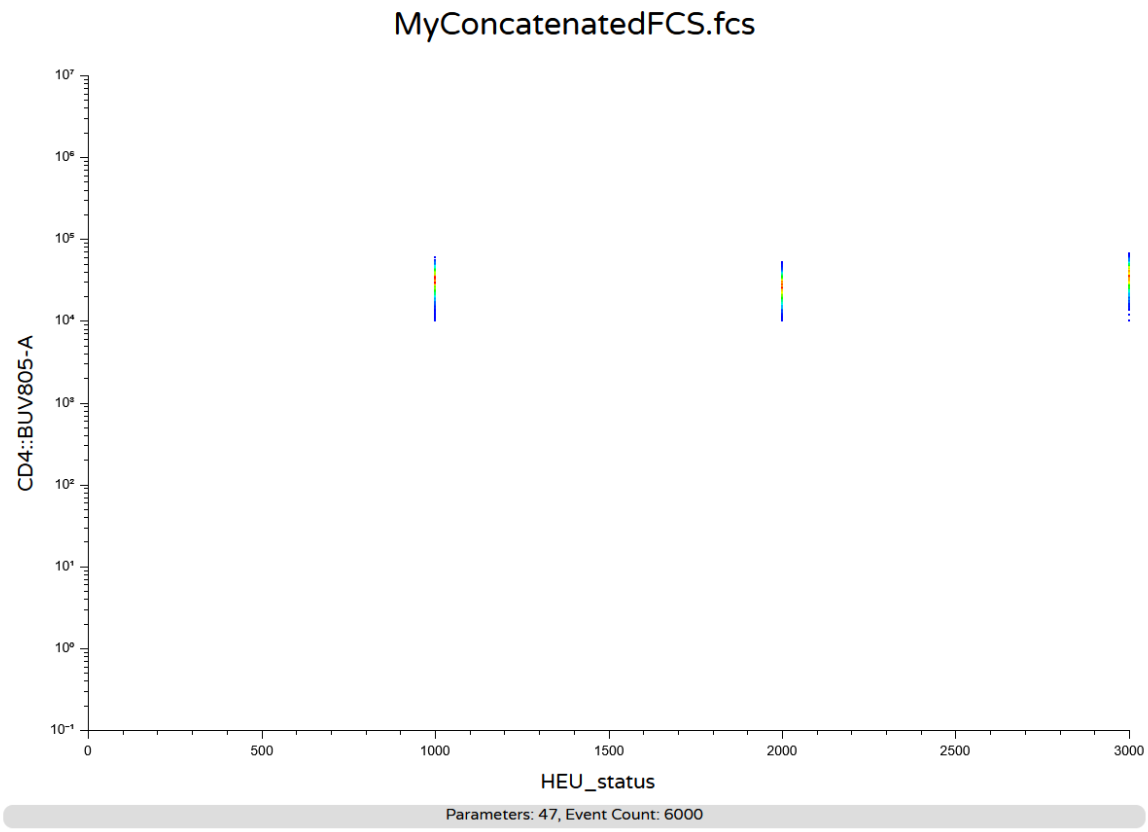
flowFrame object 'ConcatenatedCD4.fcs' with 6000 cells and 47 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P43 AF-A NA 4194275 -111 4194275 $P44 name NA 2001 1000 3000 $P45 condition NA 1 1000 1000 $P46 infant_sex NA 1001 1000 2000 $P47 HEU_status NA 2001 1000 3000 555 keywords are stored in the 'description' slot

And just as easily, we can modify returnType to return as an .fcs file

```
Concatenate(gs=SFC_GatingSet, subset="CD4+", addon="CD4",
DownsampleCount=2000, desiredCols=c("name", "condition", "infant_sex",
"HEU_status"), returnType="fcs")
```

[1] "c:/Users/Macarena/MacarenaLM/MyConcatenatedFCS.fcs" (It has been created a new fcs file)

Which, as always, for sanity check when working with your own functions, its always best to double check everything stored correctly and opens as you will anticipate.



Likewise, similar to what we encountered with Downsampling, if we provide an improbably high-number for 'DownsampleCount', we just end up combining all our cells of interest for a given cell population, from each individual, together into a single file.

```
Concatenate(gs=SFC_GatingSet, subset="CD8+", addon="CD8",
DownsampleCount=10000000, desiredCols=c("name", "condition", "infant_sex",
"HEU_status"), returnType="flowframe")
```

flowFrame object 'MyConcatenatedFCS.fcs' with 7370 cells and 47 observables: name desc range minRange maxRange $P1 Time NA 896744 0 896744 $P2 SSC-W NA 4194303 0 4194303 $P3 SSC-H NA 4194303 0 4194303 $P4 SSC-A NA 4194303 0 4194303 $P5 FSC-W NA 4194303 0 4194303 … … … … … … $P43 AF-A NA 4194275 -111 4194275 $P44 name NA 2001 1000 3000 $P45 condition NA 1 1000 1000 $P46 infant_sex NA 1001 1000 2000 $P47 HEU_status NA 2001 1000 3000 555 keywords are stored in the 'description' slot

R

Over the course of today, we created two large functions, Downsampling() and Concatenate(), plus several helper functions. While currently active in our environment, what if we want to use them for a different project on another day? What would be the best way to make them available.

One approach to handling this issue I frequently encounter is placing your own functions within individual code blocks at the beginning of your .qmd file, so that they get activated in your local environment from the start of the process. However, if the functions are lengthy, this ends up occupying substantial portion of the document, which is less than ideal.

The approach we will be using for the next several weeks of the course is to place the completed functions in their own respective .R files, all kept together in their own R folder within our working directory.

Subsequently, when we need to load all these functions into our local environment, all we need to do is provide the file paths to walk alongside source() in order to activate all the .R files, making the functions they contain within available to us.

To load all these functions to our active environment, for the next several sessions, we would only need to use the source() function providing the path to the folder. This in turn will load in all the function .R files we have created, making their contents available to us in R for subsequent use. We will explore how this approach can be useful in context over the next several weeks.

```
#RFolder <- file.path("course", "10_Downsampling", "R")
# For Interactive
RFolder <- file.path("C:/Users/Macarena/MacarenaLM/notes/
  week10-Downsampling and concatenation/R") # For Quarto Rendering
MyFunctions <- list.files(RFolder, full.names=TRUE)
purrr::walk(.x=MyFunctions, .f=source)
```

To load all these functions to our active environment, for the next several sessions, we would only need to use the source() function providing the path to the folder. This in turn will load in all the function .R files we have created, making their contents available to us in R for subsequent use. We will explore how this approach can be useful in context over the next several weeks.